MATLAB 완벽가이드



저자 : 임종수

이 문서는 한글97로 제작되었습니다. 하지만 도스용 한글3.0부터 윈도우용 한글97까지 완벽히 지원됩니다. 만약 그림부분이 나타나지 않는다면 RAM이 얼만큼인지 보시기 바랍니다. RAM용량이 적으면 나타나지 않을 수 있습니다. 또한 이 글을 보시게 될 분의 글꼴이 어떤지 모르기에 가장 기본이 되는 글꼴만 이용했음을 미리 알려드립니다.
기타 문의 사항이나 더욱더 알고 싶으신분은 다음의 연락처 혹은 홈페이지에 방문하시기 바랍니다.



하이텔 : ssljk07
E-mail : matlab@loveangel.net

제작·편집 : 임종관 「오정, 한글서당, MATLAB 전문학교, 이야기세상 운영자」
http://matlabschool.com [MATLAB 전문학교]

10월 6, 7일 이틀에 걸쳐...
임종수씨가 진행하는 세미나 개최!!

▶ 날짜와 안내

수강 인원 : 100명
세미나 날짜 : 2001년 10월 6일 (토) ~ 10월 7일 (일)
세미나 장소 : 서울 대학로 정보세미나홀 (자세한 안내도는 홈페이지에 기재)
세미나 시간 : 오전 10시 ~ 오후 6시 (12시 ~ 1시는 점심시간, 음료수 제공)
세미나 교재 : 임종수의 MATLAB 완벽가이드
(당일 시중가의 80%에 판매 - 시중가 18,000원 → 판매가 14,400원)
세미나 방식 : 5시간 강의후 2시간 질문/답변 시간

세미나 수강료 : 학생(하루 4만원, 이틀 7만원), 일반(하루 5만원, 이틀 9만원)
수강료 납부방식 : 선불이며, 무통장입금과 신용카드결제 두가지 방식으로 진행합니다.
수강신청 : http://matlabschool.com/seminar.html

▶ 강의 내용

첫째날(10월 6일 토요일)

1. matlab overview
2. matlab의 행렬 연산.
3. M file 작성 방법.
4. Matlab 그래픽스.
5. 파일입출력과 GUI programming 기본

둘째날(10월 7일 일요일)

1. Handle Graphics system.
2. Cell and struct data type.
3. 다양한 GUI Programming 방법.

※ 이번 강의는 MATLAB 5.x에 대한 것이며, 다음에는 6.x에 대한 강의를 진행할 예정입니다.
※ 이번 강의를 수강한 모든 분들께는 다음 강의를 수강료를 할인해 드립니다.



7. Matlab의 연산자(operators) :

Matlab이 지원하는 연산자(operator)에는 다른 High level언어(예, C, Pascal 등등)에서처럼 3가지 종류의 연산자가 있다.

가) 산술 연산자(Arithmetic Operators) :

수치 계산을 행하는 연산자로써, 사칙연산이나 거듭 제곱 등을 말한다.

나) 관계 연산자(Relational operators) :

배열을 구성하는 각각의 원소에 대한, "작다(less), 같지 않다(not equal to) 등"을 비교하는 데 이용한다.

다) 논리 연산자(Logical Operators) :

"AND", "OR"등의 논리 연산을 행하며 "TRUE", "FALSE"와 같은 결과를 알려 준다.

가. 연산자 우선 순위 :

산술 연산자가 가장 우선 순위가 높고, 그 다음으로 관계 연산자이며, 마지막으로 논리 연산자 순서이다. 그러나, 만일, 같은 우선 순위를 갖는 연산자이면, 왼쪽에 위치한 연산자부터 계산해 나간다. 물론, " ( ) "를 수식에 취하면 가장 높은 연산 순위를 갖는다.

나. 산술 연산자 (Arithmetic operators) :

산술 연산자 내에서의 연산 우선 순위는 다음과 같다.

1. transpose(" .' "), power (" .^ "), 복소 공액 transpose(" ' "),
   행렬의 power(" ^ ")
2. unary plus(" + "), unary minus(" - ")
3. 곱셈(" * "), 오른쪽 나눗셈(" . / "), 왼쪽 나눗셈(" . \ "), 행렬의 곱(" * "),
   행렬의 오른쪽 나눗셈(" / "), 행렬의 왼쪽 나눗셈(" \ ")
4. 덧셈(" + "), 뺄셈(" - ")
5. colon 연산자(" : ")

이제 연산자 우선 순위에 대한 예를 들어보자.

```
□A =[3 9 5];
□B =[2 1 5];
□C1=A./B.^2 ↲  % 우선, B.^2을 계산하고 이어서, A./을 계산한다.

C1 =

    0.7500    9.0000    0.2000

□C2=(A./B).^2 ↲  % 우선, 괄호 안의 식(A./B)를 계산하고 나서, .^2를 계산한다.

C2 =

    2.2500   81.0000    1.0000
```

연산자 우선 순위에 의해서 C1과 C2의 결과가 다르다는 데 주의하자.

이미, 앞에서 언급하였듯이, 행렬에 대한 산술 연산의 경우에는, 두 피 연산자인 행렬들의 크기가 같아야 한다. 그러나, 두 개의 피 연산자중 하나가 스칼라인 경우에는 그 스칼라를 다른 피 연산자의 모든 원소에 적용하여 계산한다. 이와 같이 스칼라를 행렬에 적용하여 계산할 수 있는 성질을 "스칼라의 확장성(scalar expansion)"이라 한다.



어제 후배가 필자에게 본인은 Matlab을 거의 다 아는 데 GUI coding 하는 것만 모르니 그것만 가르쳐 달라고 하였다. 그런데, 대부분의 학생은 필자의 후배처럼 Matlab 행렬 연산에 대한 충분한 지식과 경험도 없이 무리하게 포장만 하려는 경향이 있다.
또한, C언어를 잘 한다고 하여, 반드시 Matlab을 잘한다고 말할 수는 없다. 왜냐하면 C언어는 행렬을 곧바로 계산할 수 없기 때문이다. 대부분의 C언어 잘 하는 학생이 작성한 M-file을 보면, Matlab의 행렬 연산을 무시한 C언어 양식 그대로 작성한 경우가 많다. 물론, 그렇다고 프로그램이 돌아가지 않는 건 아니지만 속도 면에서 상당한 차이가 발생한다. 여러분이 필자의 강의를 순서대로 따라간다면 Matlab을 효과적으로 다룰 수 있다고 본다.

다. 관계 연산자(Relational Operators) :

관계 연산자에는 6가지 종류가 있다. 이들 6개의 관계 연산자는 같은 크기의 두 배열에 대한 해당 원소를 비교한다. 관계 연산자에서도 scalar expansion은 적용된다.

주어진 관계가 사실인 경우(TRUE) 해당 위치에는 "1"을 반환하고, 거짓인 경우(FALSE)에는 해당 위치에 "0"을 반환한다. 관계 연산자는 논리 연산자와 함께 앞으로 M-files를 작성 할 때, " **if , while, for, switch** "등과 더불어 실행순서를 제어하는 데 귀하게 쓰일 것이다.


주의

관계 연산자는 항상 원소 대 원소(element-by-element)로 연산한다.

관계 연산자의 종류	해당 연산자에 대한 설명
<	"보다 적다(Less than)"
<=	"적거나 같다(Less than or equal to)"
>	"보다 크다(Greater than)"
>=	"크거나 같다(Greater than or equal to)"
==	"같다(Equal to)"
~=	"같지 않다(Not equal to)"

표 3 Matlab의 관계 연산자들


주의

C언어에서는 "같지 않다(Not equal to)"가 " != "인 데, Matlab에서는 " ~= "이라는 것에 주의하자!

```
□A=[2 7 6;9 0 -1;3 .5 6]; ↲  % Matlab에서는 소숫점 앞의 "0"은 무시한다.
                              % 즉, "0.123"과 ".123"은 같은 뜻이다.
□B=[8 0.2 0;-3 2 5;4 -1 7]; ↲
□rel_AB= A<B ↲

rel_AB =

     1     0     0
     0     1     1
     1     0     1
```

행렬 A와 B를 비교하여, A의 원소 중 B의 원소보다 더 작은 것은 "참(TRUE)"이므로 해당 위치에 "1"을 넣고, 그렇지 않은 곳(거짓:FALSE)에는 "0"을 넣는다.
결국, 여러분은 rel_AB의 결과로부터 행렬 A의 원소 중에서 어느 원소가 행렬B의 원소보다 더 작은 지, 해당 위치를 알 수 있다.



관계 연산자는 임의의 크기를 가진 배열에 대해 적용할 수 있다. 그러나, □A == [ ] ↲과 같이 빈 행렬(empty matrix)에 대한 관계는 경고나, error를 발생한다. 만일, 여러분이 빈 행렬에 대해 테스트하고자 한다면 " **isempty** "함수를 이용하라! " **isempty** "함수는 빈 행렬인 경우 "1"을 반환한다.

```
□A=[]; ↲
□isempty(A) ↲

ans =

     1
```

이제, 관계 연산자 뿐만 아니라, 논리 연산자에서도 유용하게 사용되는 " **find** "라는 함수에 대해서 알아보자



find 함수

" **find** " 함수는 주어진 관계에 맞는 배열의 원소 위치를 알려 준다. 만일, 주어진 배열이 벡터이면 출력 변수는 한 개로 하다. 또한, 주어진 배열이 행렬이고, 출력 변수를 하나로 설정하면, " **find** " 함수는 주어진 행렬을 "열 벡터"로 변환 한 뒤, 해당 원소의 위치를 반환한다.

```
□A=magic(4) ↲  % magic 행렬은 어느 방향으로 더하던지 그 합이 같은 행렬이
               % 다.
 A =

    16     2     3    13
     5    11    10     8
     9     7     6    12
     4    14    15     1

□index=find(A > 8); ↲
□A(index)=100 ↲
A =

   100     2     3   100
     5   100   100     8
   100     7     6   100
     4   100   100     1
```

라. 논리 연산자(Logical operators) :

Matlab의 논리 연산자는 " AND, OR, and NOT "을 수행한다.

논리 연산자의 종류	해당 연산자의 설명
&	AND
|	OR
~	NOT

표 4. Matlab의 논리 연산자의 종류


주의 비트 연산자(bitwise logical operations)에 대해서는 " bitfun "디렉토리에 있으니 " help "명령어로 참고하기 바란다.

논리 연산자에 대해서도 scalar expansion은 적용된다.

1. " AND " 연산자는 두 개의 피 연산자가 참이면 "참(반환하는 값 : 1)"을 반환한다. 여기서, 참이라 함은 영이 아닌 수치(nonzero number)를 말한다.

```
□U=[-1 0 2 3 0 5]; ↲
□V=[5 6 1 0 0 7]; ↲
□U & V ↲

ans =

     1     0     1     0     0     1
```

위의 결과에서도 알 수 있듯이 "0"일 때만 거짓으로 보고, 부호와 숫자의 크기는 참·거짓과 관계없다는 데 주의하자!

2. " OR " 연산자는 적어도 한 개의 피 연산자가 참이면 "참(반환하는 값 : 1)"을 반환한다.

```
□U | V ↲
ans =

     1     1     1     1     0     1
```

3. " NOT " 연산자는 피 연산자의 부정을 행할 때 이용된다.

□~U ↲
ans =

0 1 0 0 1 0

4. 논리 함수(Logical functions) :

Matlab은 논리 연산자 외에, 여러 논리 함수를 제공한다.

논리 함수의 종류	해당 함수에 대한 설명
xor	exclusive OR연산을 행한다.
all	주어진 벡터의 모든 원소가 "참(nonzero)"인 경우에만 "1"을 반환한다.
any	주어진 벡터의 임의의 원소가 "참(nonzero)"인 경우에 "1"을 반환한다.
isnan	주어진 벡터의 원소 중에서 "NaN"이 위치한 곳에는 "1"을 반환한다.
isfinite	주어진 벡터의 원소 중에서 유한한 크기를 갖는 수치, 또는 NaN이 아닌 곳에는 "1"을 반환한다.
isinf	주어진 벡터의 원소 중에서 유한한 크기를 갖는 곳에는 "1"을 반환한다.

표 5. Matlab의 논리 함수들

[표 5]의 논리 함수 중에서, " **all** "함수와 " **any** "함수는 특별히 중요한 함수이다.
만일, " **all** "함수와 " **any** "함수에 주어진 입력 변수가 벡터가 아니고, 행렬이면 각각의 " **열 방향** "에 대해서 연산한다.

□U=[1:4 0]; ↲
□all(U) ↲

ans =

0

□A=[0 1 2;3 5 0] ↲
A =
0 1 2
3 5 0

```
□all(A) ↲
 ans =

     0     1     0    % 행렬 A의 각각의 "열 방향"으로 논리 계산을 행한 데 주의!

□any(A) ↲

ans =

     1     1     1
```

지금까지 여러분은 Matlab의 행렬연산에 대하여 많은 것을 공부하였다. 비록, 여러 부분이 빠지긴 했지만, "Matlab 행렬 연산II"를 따로 분류하여 이곳에서 설명하지 않은 "다항식 계산과 데이터 분석 등"은 뒤에서 다룰 것이다.

이제, 단원을 바꿔서 "2. M-file Programming"에 대하여 다루겠다.

2 M-file Programming

I. M-file이란 무엇인가?

전체적인 의미로 M-file이란 Matlab언어로 쓰여진 파일들(files)을 말한다.
또한, M-file은 연속적인 Matlab문장들(명령어들)을 수행하는 script 양식(script mode)과 입력 매개변수와 출력 매개변수를 다루는 함수 양식(function mode)이 있다. 이와 같은 M-file은 Matlab이 제공하는 text editor를 이용하여 작성 할 수 있고, 또한, 다른 여러 종류의 ASCII text editor를 이용할 수도 있다. 그러나, 앞으로의 M-file 작성은 모두 Matlab이 제공하는 text editor를 이용하여 작성 할 것이다.
Matlab에서 M-file의 이용과정은 다음과 같다.

1. text editor를 이용하여 M-file을 작성한다.
2. command window에서 M-file을 부르거나, 또는 다른 M-file내에서 호출 할 수 도 있다.

[표2.1]은 script mode의 M-file과 function mode의 M-file에 대한 비교이다.

script mode의 M-file	function mode의 M-file
파일 이름은 어떠한 이름으로도 할 수 있다.	파일의 이름은 function이름과 가능 한한 같게 한다.
base workspace에 있는 data에 대해서 연산한다.	function workspace에 있는 data에 대해서 연산한다.

표 2.1 M-file 종류 비교

[표2.1]에서 base workspace란 Matlab command window에 있는 변수 공간을 의미한다. 즉, 지금까지 여러분이 [그림 1.5]에 보여준 Matlab command window에서 생성한 변수들로써, [그림1.9]에 나타나는 변수(A, ans, z 따위)를 의미한다. 그리고, function workspace란 여러분이 작성한 function mode의 M-file내에서만 존재하는 내부 변수를 의미한다. 물론, 전역 변수나, userdata를 이용하면, function workspace 변수를 base workspace에서도 볼 수 있다. 그러나, base workspace와 function workspace의 보다 편리한 데이터 교환은 출력 변수와 입력 변수를 이용하는 것이다. Matlab은 이처럼 변수를 연산하는 작업공간이 따로 설정되어 있다는 것을 알아두자! 일반적으로 그냥, workspace라고 하면 " **base workspace** "를 의미한다.

Matlab 4.2c이전 버전들에서는, M-file mode에 따라 저장하는 파일 이름에도 차이가 있었다. script mode의 M-file은 어떠한 이름으로도 파일 이름을 설정할 수 있지만, function mode의 M-file은 function이름과 같은 이름의 M-file로 저장해야 했었다. 그러나, Matlab5.0 이후 버전에서는 이러한 제한요소가 없어졌다. 만일, 함수 정의 부에 있는 이름과 파일 이름

이 다르다면, Matlab은 함수 정의 부에 있는 이름을 무시하고, 파일이름을 이용한다. 그러나, 가능한한 function mode M-file의 경우, 파일 이름은 함수이름과 같게 하는 것이 좋겠다.
이제, function mode M-file과 script mode M-file의 예를 각각 살펴보자!

II. 간단한 function mode M-file :

[그림 2-1]의 " **New File** " icon을 클릭 한다.



그림 2-1. command window toolbar

" **New File** "icon을 클릭 하면, [그림 2-2]와 같은 " **Matlab editor/debugger** " text editor 가 뜰 것이다.

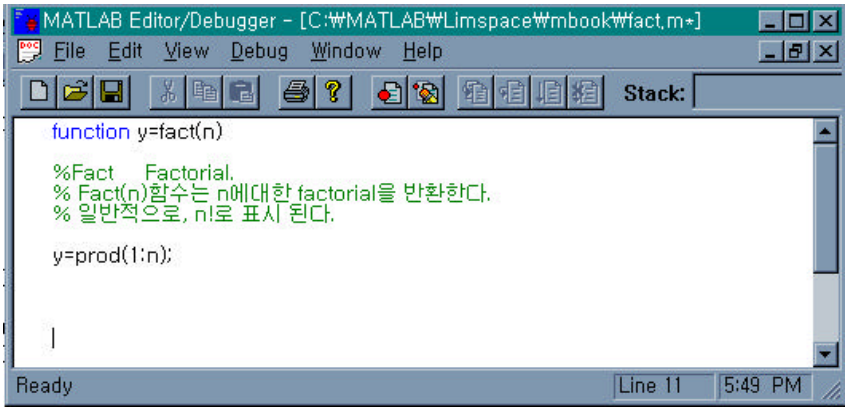


그림 2-2 Text editor/debugger

[그림 2-2]에서 작성한 M-file은 " **n!** "을 구하는 함수이다.
Matlab에서 function mode M-file을 작성하려면,

가. " function 출력 변수 = 함수 이름(입력 변수) "의 형태로 함수를 정의한다.

나. " % "으로 시작하는 주석을 단다.

→ 함수 정의 부(part) 바로 다음에 나오는 첫 번째 주석 문은 " **Lookfor** " 를 이용한 M-file의 검색시 나오는 정보 부분이다. (" **lookfor** "에 대한 내용은 page.15 참고)

다. " Help "명령 다음에 나오는 정보들은 함수 정의 부분과 첫 번째 비 주석문(즉, Matlab 명령어 또는 빈 공간(blank))사이에 있는 주석문들 이다.

[그림 2-2]의 경우에서는 "**%FACT 표시 된다.**"까지가 "**help fact**"할 때 나 오는 정보 내용이다.

라. " y=prod(1:n) "은 함수의 본체이다.

→ " prod "함수에 대해서는 page. 20을 참고,

가 라. 는 function mode M-file작성 시, 기본적인 부분이다.

이제, M-file을 다 작성했으면, 함수이름인 " fact "로 파일 이름을 정한 뒤 저장한다. 물론, 확장자는 " .m "이다.



Matlab4.2c이전 버전에서는 함수나 변수 이름이 8자 이상이 되는 것을 허용하지 않았다. 그러나, Matlab5.0이후 버전에서는 변수나, 함수 이름을 31자 이내에서 만들 수 있게 되었다. Matlab에서도 다른 언어에서와 마찬가지로 변수나 함수이름의 첫 글자는 문자(letter)이어야한다.

[표2.2]는 Matlab에서 자주 쓰이는 명령어들을 간단히 설명하고 있다. 보다 자세한 이용법은 앞으로 배워나갈 것이다.

Matlab 명령어	명령어에 대한 설명
close 해당 그림의 핸들	활성화된 그림을 지운다. →" close all "하면 모든 그림이 지워짐!
clear 변수 이름	base workspace에 있는 변수들을 지운다. →" clear all "모든 변수와 핸들이 지워짐
dir	"DOS"에서의 "dir"과 거의 같은 기능을 한다. →wildcards이용 가능!
clc	"DOS"에서의 "cls"와 같은 기능! →변수와 핸들에는 영향이 없음

표 2.2 Matlab의 주요 명령어들

" dir "대신 "UNIX"운영 체계에서 쓰는 " ls "를 사용해도 command window의 내용을 다 리스트 한다. 그러나, 역시, 변수와 핸들에는 영향을 주지 않는다.

" 핸들(handle) "에 대해서는 뒤에서 따로 단원을 분리하여 " 객체(object) "와 함께 설명하겠다. 본 단원에서는 핸들의 정의와 기본적인 이용법만 다루겠다.

만일, 여러분이 앞에서 설명한 " startup.m "내의 디렉토리를 그대로 기본 작업 디렉토리로 하고, 부가적으로 또 다른 작업 디렉토리를 설정하고 싶으면, 다음과 같이 행한다.

1. 원하는 위치에 작업 디렉토리를 만든다.
   → 여기서는, " c:\Matlab\limspace\mbook "으로 정하였다.
2. [그림 2-1]의 command window toolbar에서 "path browser" icon을 클릭 한다. 그러면 [그림 2-3]과 같은 "Matlab path" window가 뜰 것이다.
3. "Matlab path" window내의 "add to path..." 버튼을 클릭 한다.
   그러면 [그림 2-4]와 같은 "add to path" window가 뜰 것이다.
4. 새로 만든 디렉토리(여기서는 "mbook")를 클릭 한 후, "add to front" 또는,
   "add to back" 둘 중 하나를 클릭 한다. 그리고 나서, "OK" 버튼을 클릭 한다.

5. “**Matlab path**" window내의 ”**save settings**" 버튼을 클릭 한다. 그러면,
“**path**"라는 title안에 여러분이 새로 만든 디렉토리가 Matlab의 다른 여러 디렉토리와 링크된(linked) 것을 알 수 있을 것이다.

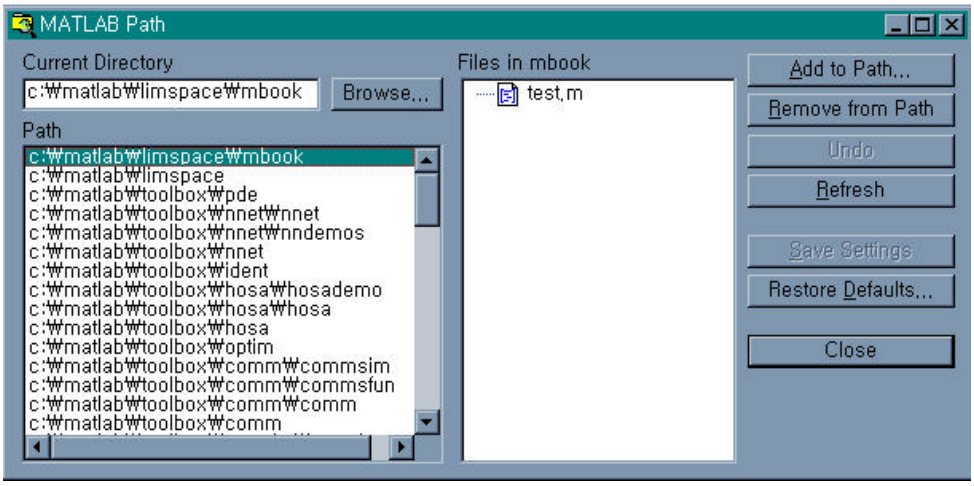


그림 2-3 “Matlab path" window

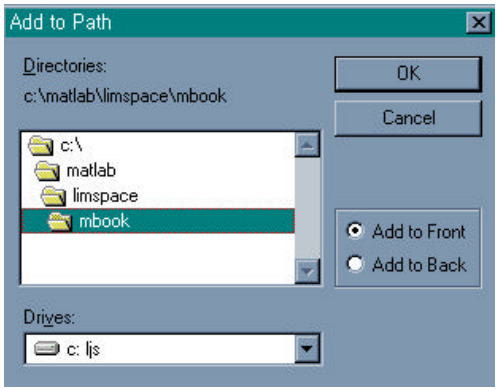


그림 2-4. “add to path" window

[그림 2-5]는 command window에서 “ **fact.m** "을 실행한 예를 보여 주고 있다.
[그림 2-2]에서 알 수 있듯이, “ **fact.m** "함수는 적당한 입력 매개변수 값을 주면, 그 입력 매개변수에 대한 factorial을 계산한다. 그리고, 그 결과를 출력 매개변수를 통하여 base workspace로 반환한다.
→ [그림 2 - 5]는 함수“ **fact.m** "이 “ 3! =6 ”을 계산한 결과이다.

[그림 2-5]에서 알 수 있듯이, 함수에 보내주는 이름은 [그림 2-2]에서 보여준 입력 매개변수 이름(즉, “ **n** ")과 달라도 된다.

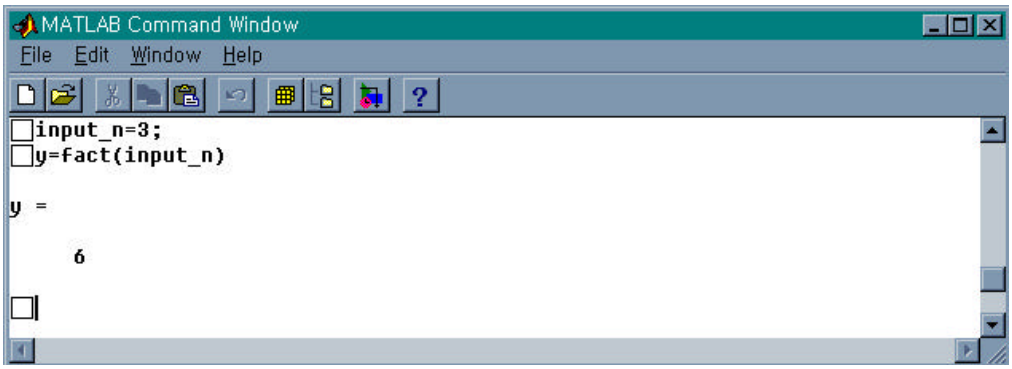


그림 2-5 "fact.m"함수의 실행 예

III. 간단한 script mode M-file :

function mode M-file과 달리, "script mode M-file"내에 있는 모든 변수는 "base workspace" 메모리 공간에 저장된다. 즉, script mode M-file내에 있는 변수는 command window내에 있는 변수와 데이터 교환이 가능하다. 그러므로, "text editor"에 간단히 "y=prod(1:n)"이라고 한 뒤, "script_fact.m"으로 저장하고, command window에서 [그림 2-6]과 같이 실행하면, 위에서 얻은 "fact(3)"와 같은 결과를 얻는다.

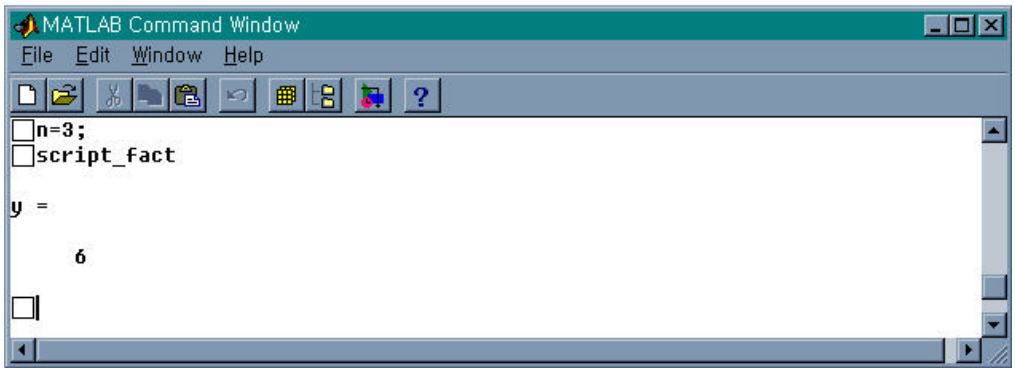


그림 2-6 "script_fact.m"의 실행 예

"script_fact.m" 파일을 실행한 뒤 " whos "명령어를 실행하면, base workspace에 " n, y " 변수가 새로 생긴 것을 알 수 있을 것이다.

IV. 명령어의 흐름 제어(Flow Control) :

Matlab5.0에는 명령어의 흐름을 제어하는 4가지 방법이 있다.

가. if, else와 elseif :

전형적인 사용법(syntax) :

```
if 논리적인 식
        명령어 문장
   elseif 논리적인 식
        명령어 문장
else
        명령어 문장
end
```

[그림 2-7] " if문장 "의 기본 형태

만일, "if 문장"의 논리적인 식이 "1(TRUE)"이면, Matlab은 명령어를 수행한다. 그러나, 논리적인 식의 값이 "0(FALSE)"이면 명령어 문장을 건너뛴다.(skip)



참(TRUE)과 거짓(FALSE)에 대해서는 이미 앞에서 자세히 언급하였다.

[그림 2-7]에서 "elseif"와 "else"명령어의 차이점은 논리적인 식을 갖는가 갖지 않는가 하는 점이다.



만일, 논리적인 식의 값이 scalar가 아니라 배열(array)로 주어진다면, 그 배열의 모든 원소가 "참(TRUE)"이어야 "참(TRUE)"이다. 즉,

```
if 배열
   명령어 문장
end
```

(a)의 경우

```
if all(배열(:))
    명령어 문장
end
```

(b)의 경우

(a)의 경우와 (b)의 경우는 같다. 또한, 논리적인 식의 값이 빈 행렬(empty matrix)이면 "거짓(FALSE)"에 해당한다.

[예제 1] 벡터 x의 평균을 구하라!

```
function y=average(x)

%AVERAGE   Mean of vector elements.
%average(x)는 벡터 x의 평균을 반환하며,
%만일, 입력이 없으면 error를 내보낸다.

[m,n]=size(x);
if (  ((m==1) | (n==1)) | (m==1 & n== 1))
   error('입력은 값이 있는 벡터여야 한다.');
end

y=sum(x)/length(x);  %실제, 계산하는 부분!
```

[그림 2-8] "average.m"파일의 내용

[그림 2-8]에서 "**error('메세지')**"함수는 "메세지"내용을 screen에 내보낸(display) 뒤 해당 함수의 실행을 멈추고, base workspace로 연산을 옮긴다. 여기서, 주의 할 점은 "**string**"을 표시하기 위해서 단일 인용부호(즉, " ' ' ")가 쓰였다는 데 주의하자! C언어에서는 "string"을 표시하는 데, 이중 인용부호를 사용한다는 점에서 다르다. 유사 함수로 "**warning('메세지')**"함수가 있다.

[그림 2-8]에 있는 내용은 쉽게 이해가 가리라고 본다.



만일, 여러분 중에서 [그림 2-8]의 M-file내용을 작성하기 싫다고 해서, 내용을 copy한 후 Matlab의 text editor에 paste하면 아마, 낭패를 당할 것이다. Matlab은 한글로 된 명령어를 받으면 치명적인 error에 빠진다. 필자가 Matlab4.2c를 사용할 때 였다. 실수로 command window에 한글로 명령어를 작성하고, return key를 누르자 화면이 정지하고, 아무 반응이 없는 것이었다.
"Ctrl-Alt-del"key를 눌러도 시스템에 반응이 없었다. 결국, "reset" switch를 누른 후 다시 Matlab을 시동시키자 이번에는 속도가 무척 떨어졌었다. [그림 2-8]에도 한글로 작성된 한 개의 문자가 있다. 그러니, 꼭 여러분 손으로 직접 작성하기 바란다.

[그림 2-9]는 "average.m"함수를 실행한 예이다.

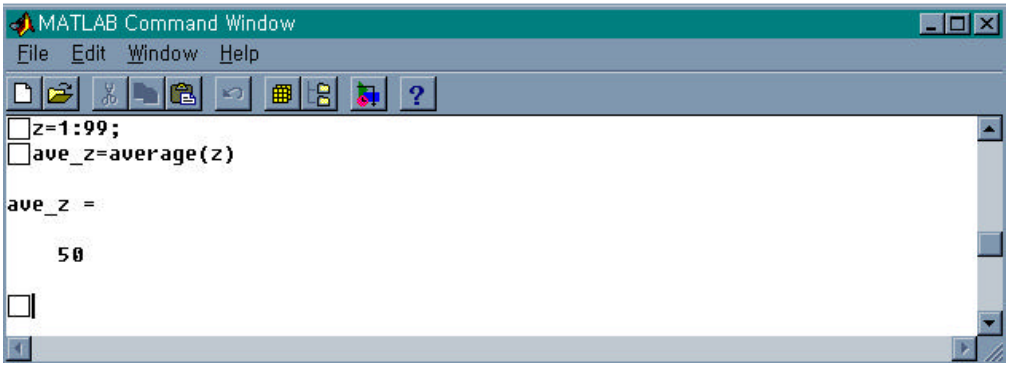


그림 2-9. "average.m"을 실행한 예

나. switch 문 : Matlab5.0이후 버전에만 있다.

전형적인 사용법(syntax) :

```
switch 변수 또는 식
        case 값1%만일, 변수 또는 식이 "값1"인 경우!
            명령어 문장
        case 값2%만일, 변수 또는 식이 "값2"인 경우!
            명령어 문장

        ... 임의의 수의 case 문들 ...

        otherwise       %만일, 변수 또는 식의 값이
                        %어떠한 case 값들과도 맞지 않을 때
            명령어 문장
end
```

[그림 2-10] "switch 문"의 기본 형태

C언어에서의 "**switch** 문"과는 달리 Matlab에서의 switch문은 변수 또는 식의 값과 맞는 "case문"의 명령어만 수행하므로 C언어에서처럼 "**break**"명령어가 따로 필요 없다. 또한, C언어에서의 "**switch**"문에 있는 "**default**"명령어는 Matlab에서 "**otherwise**"에 해당한다.

[예제 2] 입력 변수 x의 짝수·홀수를 판별하는 M-file을 작성 할 것.

[풀이]
간단하게 입력 변수 "x"의 값이 "2"로 나누어지면, 짝수이고, 그렇지 않으면, 홀수로 판단하는 알고리즘을 생각할 수 있다. 또한, 각각의 결과를 메시지로 내보내는 M-file을 작성할 수 있다.

다음은 M-file 작성 시, 필요할지도 모를 함수를 정리하였다.

Matlab 함수들 :
"**disp('메세지')**"함수는 단순히 screen에 "**메세지**"를 내보내는 기능을 한다.
"**rem(a , b)**"함수는 "a ÷ b"에 대한 나머지를 반환한다.
"**fix(x)**"함수는 "x"가 소수인 경우, 소수점 이하를 버린 정수를 반환한다.

이제, 여러분은 앞에서 배운 "**error**"함수를 이용하여 M-file을 작성 할 수 있을 것이다. [그림 2-11]은 필자가 작성한 M-file이다.

```
function PM_c(x)

%PM_c(x) 짝·홀수를 판별하는 M-file.
%
%1997.1.25       By Lim Jong_Su


if x ~=abs(x)
   error('입력 값은 양의실수이어야 한다.!');
elseif x ~= fix(x)
   error('정수가 아니다.');

end

y=rem(x,2);

switch y
case 0
   disp('입력 값은 짝수!');
otherwise
   disp('입력 값은 홀수!');
end
```

[그림 2-11] "PM_c.m"파일의 내용

다. for 문 :

전형적인 사용법(syntax) :

```
for 지수(index)=초기치 : 증분 : 최종치
        명령어 문장들
end
```

[그림 2-12] "for 문"의 기본 형태

기본적으로 증분량을 지정하지 않으면, 매번 반복할 때마다 증분하는 양은 "1"로 된다.



Matlab은 배열의 첫 번째 요소를 다른 언어에서처럼 "0"으로 잡는 것이 아니라 "1"로 잡는다. 그러므로 초기치가 "0"부터 시작한다면 배열의 subcript로는 사용될 수 없다.(배열에 대한 subscript에 대해서는 이전에 자세히 설명하였다.)

[예제 3] 선형상계수미분 방정식(Linear, Constant coefficient, Ordinary equations)에 의해 표현되는 시스템은 다음과 같이 간단히 표시 될 수 있다.

$$dx/dt = Ax \qquad (2\text{-}1식)$$

여기서, $x = x(t)$ 는 "t(즉, time)"의 함수이다. 그리고, A는 "t"에 독립적인 행렬이다.
식(2-1)로 표현되는 시스템의 해(solution), x(t) ,를 구하라.

단, 계수 행렬 $A = \begin{bmatrix} 0 & -6 & -1 \\ 6 & 2 & -16 \\ -5 & 20 & -10 \end{bmatrix}$, 초기조건 $x(0) = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$ 이라고 하자.

[풀이]
약간은 어려운 문제일지도 모른다. 하여튼, (2-1)식은 공업 수학 앞부분을 보면 쉽게 풀을 수 있다.

$$x(t) = e^{tA}x(0) \qquad (2\text{-}2식)$$

식(2-2)식은 행렬 지수(matrix exponential)로 표현된다.
여기서, "**expm(a)**"함수는 행렬지수(matrix exponential)를 계산하여 반환하는 함수이다.
(page. 26 참고)

"**plot3**"는 3차원 데이터를 그려주는 함수인데 여기서는 간단한 사용법만 알아보고 다음 단원에서 모든 그래픽을 자세히 알아보자.

전형적인 사용법(syntax) : **plot3.m**
"**plot3(x,y,z,'linestyle')**"
→"**x, y, z**"에 각각의 데이터를 넣는데, 같은 크기의 배열이어야 한다.
→'**linestyle**'에는 여러분이 원하는 선의 모양(style)과 색을 정하여 →지정한다.

이제, 시간 구간 $0 \le t \le 1$ 사이에서 101점(point)을 취하여 해(solution), x(t),를 구하는 M-file을 작성하면 [그림 2-13]과 같다.

```
%
%이 프로그램은 [예제 3]에 대한 내용들이다.
%
%By MathWorks.com
%-->free program.

close all
clear all

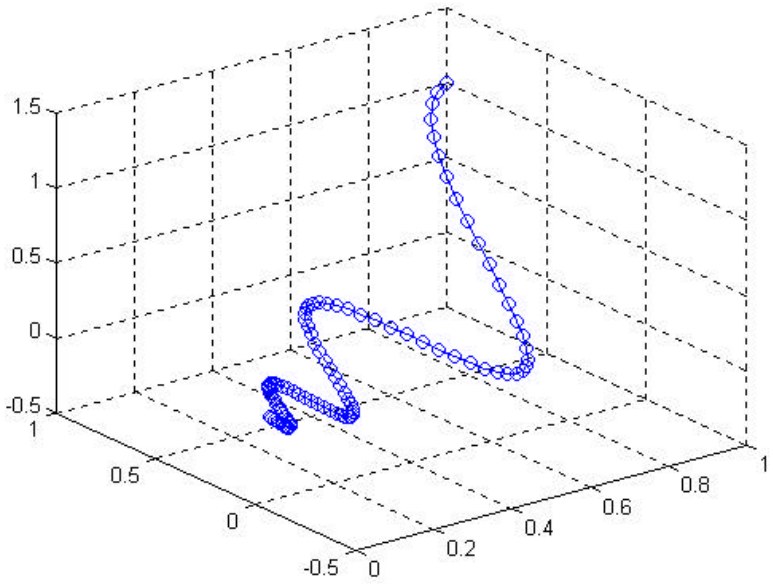
X=[];
X0=[1 1 1]';
A=[0 -6 -1;6 2 -16;-5 20 -10];

for t=0:.01:1
   X=[X expm(t*A)*X0];
end

plot3(X(1,:),X(2,:),X(3,:),'-o')
grid on
```

[그림 2-13] "ex3_sys.m"에 대한 내용

이제, "ex3_sys.m"을 실행하여 보자!



[그림 2-14] "ex3_sys.m"을 실행 한 예

[그림 2-13]의 내용에서 맨 아랫줄에 있는 "**grid on**"은 [그림 2-14]에서처럼 격자(grid)를 첨가하라는 뜻이다.

라. while 문 :

전형적인 사용법(syntax) :

```
while 논리적인 조건
        명령어 문장들
end
```

[그림 2-15] "while 문"의 기본 형태

논리적인 조건이 "참(TRUE)"이어야 명령어 문장을 실행한다. "**while 문**"에서의 "참일 조건"은 "**if 문**"에서의 "참일 조건"과 같으니 중복 설명은 하지 않겠다.


주의 Matlab에서도 C언어에 있는 "**break**"명령어가 있는 데, 기능은 서로 같다고 보아도 된다.

[예제 4] 100보다 작은 첫 번째 "n!"의 "n"을 구하라

[풀이]
역시, "prod.m"함수를 이용하자.

```
n=1;

while prod(1:n) < 100
   n=n+1;
end

result=n-1
```

[그림 2-16] "ex4.m"에 대한 내용

V. M-file의 debugging 방법 :

Matlab4.2c이전 버전에서 debugging하는 과정은 command window에서 일일이 해당 debug명령어를 typing해야 했었다. 그러나, Matlab5.0이후 버전에서는
"**Matlab editor/debugger**"window에서 간편하게 수행할 수 있게 되었다.

우선, 간단한 예제 프로그램을 작성 한 후 시작 해본다.

[예제 5] 다음과 같은 식으로 표현되는 신호를 "sinc"신호라고 한다.

$$y = \begin{cases} 1, & t = 0. \\ \dfrac{\sin(\pi * t)}{(\pi * t)}, & t \neq 0 \end{cases} \qquad \text{(2-3식)}$$

여기서, "t"는 시간을 의미하는 데, 시간 구간은 $-2 \le t \le 2$이고, 샘플링 비율(sampling rate)은 1/1000이라고 한다. 결과를 그림으로 나타내어라!

(2-3식)의 결과를 그리기 위해서는 "plot.m"함수를 이용하여라.

전형적인 사용법(syntax) : **plot.m**
"plot(x,y,'linestyle')"
→"x, y"에 각각의 데이터를 넣는데, 같은 크기의 배열이어야 한다.
→'linestyle'에는 여러분이 원하는 선의 모양(style)과 색을 정하여
→지정한다.

[풀이]
"for 문"과 "if 문"을 적당히 사용하면 쉽게 coding할 수 있을 것이다.
파일 이름은 "msinc2.m"이라고 정하였다.

[그림 2-17]은 M-file의 내용을 보여주고 있다.

```
%
%SINC   sinc신호를 만들어 내는 M-file.
%
%1997.1.25          By Lim Jong_Su
%

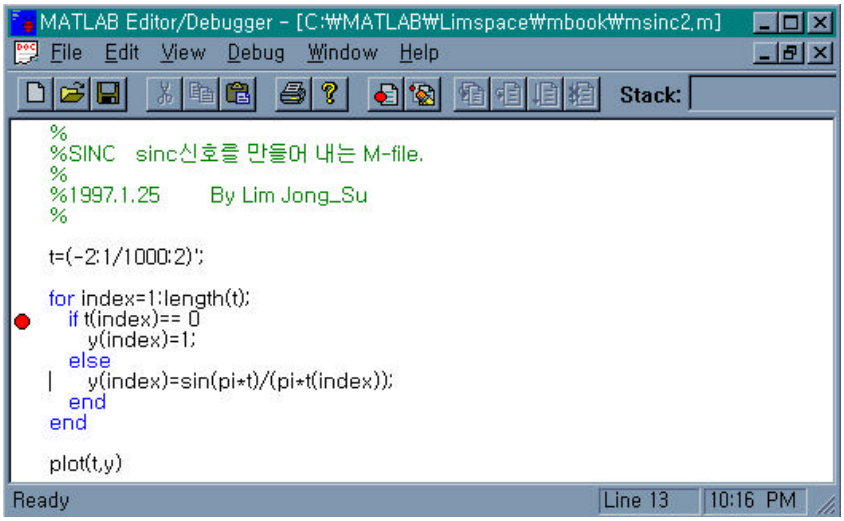
t=(-2:1/1000:2)';

for index=1:length(t);
   if t(index)== 0
      y(index)=1;
   else
      y(index)=sin(pi*t)/(pi*t(index));
   end
end

plot(t,y)
```

[그림 2-17] "msinc2.m"함수의 내용

그런데, [그림 2-17]에는 잘못 작성한 부분이 있다. 물론, M-file을 자주 만들어 본 독자라면 쉽게 잘못 된 부분을 찾아 낼 수 있을 것이다. 그러나, 잘 못된 부분이 어딘지 잘 모르는 경우, debugging을 해 본다. 우선 그림 [2-18]과 같이 "**Matlab Editor/debugger**" window에 있는 "**set/clear breakpoint**" icon을 클릭하여, 의심이 가는 부분에 "**breakpoint**"를 정한다.



[그림 2-18] "Editor/debugger"를 이용한 debugging 과정

"**breakpoint**"를 설정하였으면 command window에서 해당 M-file을 실행한다.

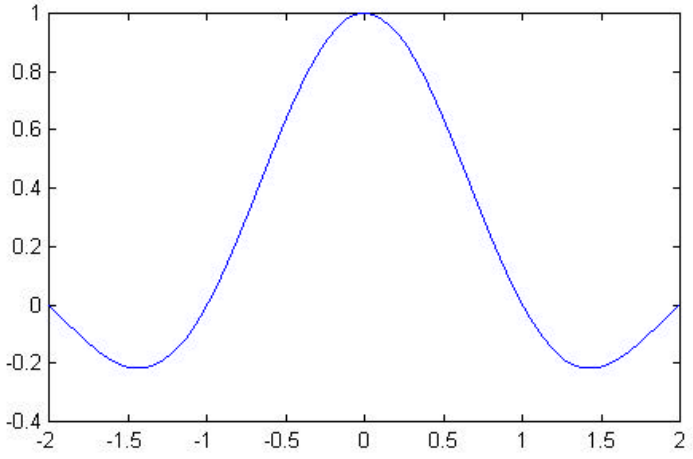
□msinc2 ↲

그러면, command window에 나타나는 Matlab prompt(즉, "□")가 "k□"로 바뀔 것이다. 이제, "**step in, single step, continue, quit debugging**"과 같은 debugger 명령어를 이용하여 debugging을 해나가면 된다. Visual C++보다는 상당히 원시적인 방법이긴 하지만 그런데로 쓸만하다.

"**single step**" icon을 이용하여, debugging을 계속해 나가다 보면, "Line 13"에서 "**error**"가 발생하는 것을 알 수 있을 것이다.

사실, "line 13"은 "**y(index)=sin(pi*t(index))/(pi*t(index));**"이 되어야 한다.

[그림 2-19]는 "msinc2.m"의 실행 결과를 보여 주고 있다.



[그림 2-19] "msinc2.m"의 실행 결과

VI. M-file의 실행 속도 비교 :

Matlab에서는 가능한 한 반복 loop문들(즉, "while, for 문")을 사용하지 않는 것이 좋다. 왜냐하면, Matlab은 다른 High-level 언어보다 반복 loop문장에서 속도가 상당히 떨어지기 때문이다. 만일, 여러분에게 [그림 2-17]의 "msinc2.m" 파일에서 반복 루프를 사용하지 말고 다시 작성 해보라고 하면 할 수 있겠는가? 만일, 쉽게 할 수 있다면 여러분은 이미 Matlab의 행렬연산에 대해 상당히 익숙해 졌다고 봐도 된다.

반복 루프를 사용하지 않고 배열만 사용하면, Matlab의 실행 속도는 반복 루프를 사용할 때 보다 거의 10배 이상 더 빨라진다. 심지어 실행 파일(즉, *.exe 등등)보다도 더 빨라진다. [그림 2-20]은 [그림 2-17]의 반복 루프를 없앤 경우의 M-file로 실행 결과는 서로 같다. 파일 이름은 "**msinc.m**"이다.

```
%
%SINC   sinc신호를 만들어 내는 M-file.
%--->배열만 이용한 경우!
%
%1997.1.25          By Lim Jong_Su
%


t=(-2:1/1000:2)';
y=zeros(size(t));

t_pos=find(t==0);
t(t_pos)=[];

y(1:end-1)=sin(pi*t)./(pi*t);

y(t_pos+1:end)=y(t_pos:end-1);
t(t_pos+1:end+1)=t(t_pos:end);

t(t_pos)=0;
y(t_pos)=1;

plot(t,y)
```

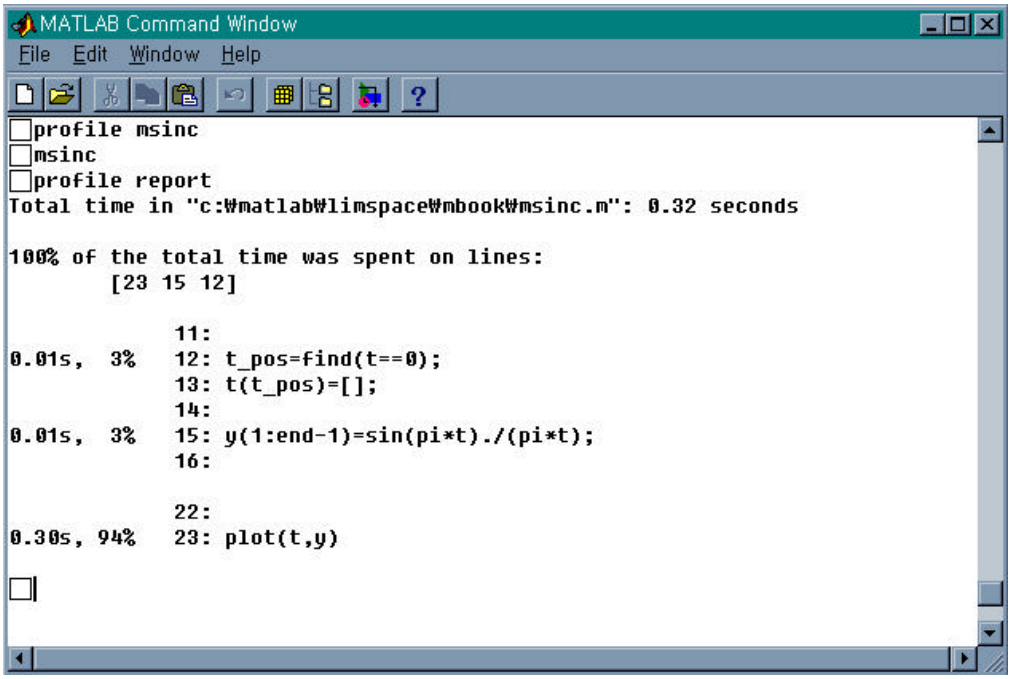
[그림 2-20] "msinc.m"의 내용

[그림 2-20]의 내용이 쉽게 이해가 간다면, 과연, [그림 2-17]의 "msinc2.m"이 [그림 2-20]의 msinc.m"보다 얼마나 느린가 알아 볼 필요가 있다.

M-file의 실행 속도를 알아보려면 "**profile.m**"이라는 함수를 이용하면 된다.

전형적인 사용법(syntax) : **profile.m**
"**profile 키워드**"
→키워드가 "**파일 이름**"인 경우, profile을 가동할 준비를 한다.
→키워드가 "**done**"인 경우, profile의 기능을 정지시킨다.
→키워드가 "**report**"인 경우, 주어진 M-file의 각 라인에 대한
→속도를 요약한 정보를 보여 준다.

[그림 2-21]은 "**msinc.m**"에 대해 "**profile.m**"의 실행한 결과를 보여 주고 있다.



[그림 2-21] "msinc.m"에 대한 "profile.m"의 실행 결과

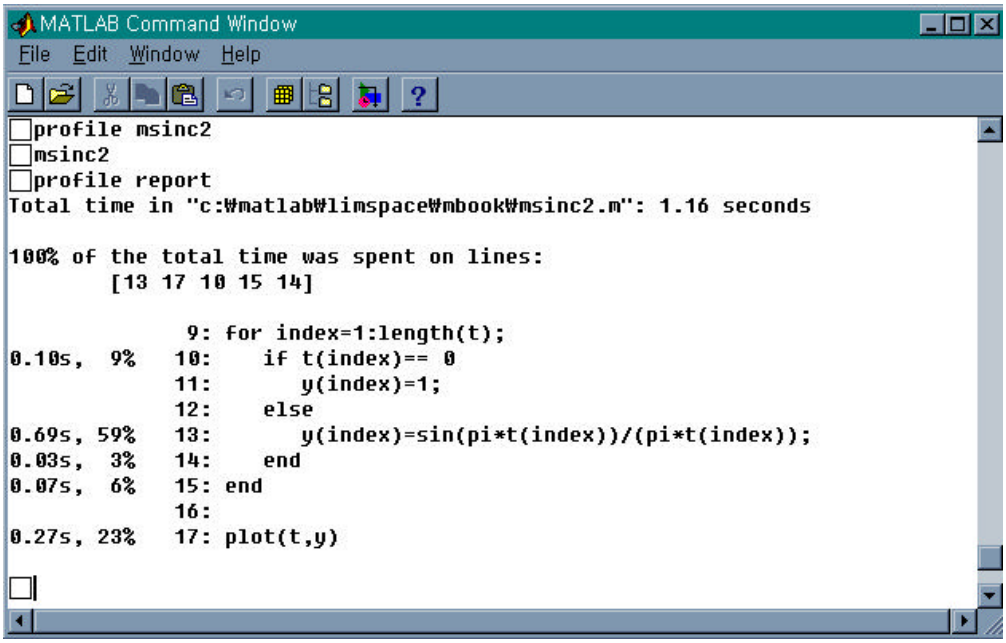
[그림 2-21]에서 알 수 있듯이 "msinc.m"의 실행 소요 시간은 "**0.32초**"이다. 또한, 각각의 라인마다 소요되는 시간을 "초"와 "백분율"로 나타내고 있다.


주의

[그림 2-21]에 나오는 소요 시간들은 사용하는 시스템에 따라 다르다.

[그림 2-22]는 "msinc2.m"에 대한 "**profile.m**"의 실행한 결과를 보여 주고 있다. 그림에서 알 수 있듯이 대부분의 시간이 반복 루프에서 소요되고 있다.

결국, 반복 루프를 사용하지 않는 것이 훨씬 능률적이라는 것을 알 수 있을 것이다.
Matlab을 더욱 효율적으로 사용하려면 최대한 반복 루프를 이용하지 않는 것이 좋다.



[그림 2-22] "msinc2.m"에 대한 "profile.m"의 실행 결과

VII. 서브 함수(subfunctions) : matlab5.0이후 버전에만 있음.

matlab5.0이후 버전부터는 다른 High_level언어에서처럼 서브함수(subfunctions)를 정의할 수 있게 되었다. function mode M-files의 첫 번째 함수를 "**primary function**"이라 하고, 또한, primary function의 이름이 해당 M-files의 파일 이름이 된다. 그리고, primary function 안에 새롭게 정의된 함수들을 "**subfunctions**"라고 한다. subfunction은 해당 primary function안에서, 또는 같은 파일 안에 있는 subfunction에서만 참조가 가능하다.

각각의 subfunctions 또한, 자신만의 함수 정의 부를 소유 해야한다.

[예제 6] 벡터 in의 평균(average)과 중간 값(median value)을 구하는 함수를 작성하라.

[풀이] 이전에 벌써 평균을 구하는 coding은 충분히 배웠고, 또한, 여러분 실력으로 볼 때, 중간 값을 구하는 coding도 어렵지 않을 것이라고 본다.

[그림 2-23]은 해당 coding을 보여 주고 있다.

```
function [avg,med]=newstats(in)

%NEWSTATS  서브 함수를 이용하여 평균과 중간 값을 구하는 함수.

n=length(in);
avg=newmean(in,n);
med=newmedian(in,n);

%서브 함수들.
function a=newmean(in,n)
%평균을 계산 한다.
a=sum(in)/n;

function m=newmedian(v,n)
%중간 값을 계산한다.

w=sort(v);
if rem(n,2)==1
   m=w((n+1)/2);
else
   m=(w(n/2)+w(n/2+1))/2;
end
```

[그림 2-23] "newstats.m"파일의 내용

가. 전역 변수(global) :

앞에서 설명하였듯이, M-files는 mode에 따라서 각각의 workspace를 가지고 있다. 즉, command prompt에서 사용되는 base workspace와 function mode에서 사용되는 function workspace가 있다. 또한, 임의의 mode에서 정의된 변수는 지역변수로써 해당 workspace에서만 참조가 가능하다. 그러나, 만일, 함수간에 또는, 서로 다른 workspace간에 데이터를 교환하기 위해서는 입·출력 매개변수를 이용하거나 본 절에서 설명할 전역 변수를 이용해야 한다.

전역 변수(global)를 사용하기 위해서는 해당 workspace에서 변수를 "**global**"로 선언하면 된다.

구체적으로 여러분의 M-files에서 전역 변수를 사용하는 방법은 다음과 같다.

1. function workspace간의 데이터 교환 시:

각각의 해당 function의 정의 부 아래에 "**global 변수 이름들**"의 형태로 전역 변수를 선언한다.

2. function workspace와 base workspace간의 데이터 교환 시 :

각각의 해당 function의 정의 부 아래에 "**global 변수 이름들**"의 형태로 전역 변수를 선언하고, command prompt에도 "**global 변수 이름들**"의 형태로 참조하고자 하는 변수를 전역 변수화 한다.



대개의 전역 변수는 대문자로 쓰는 것이 통례인데, 이때, 함수의 이름은 대·소문자를 구별하지 않지만, 변수의 이름은 대·소문자를 구별한다는데 주의한다.

[예제 7] [그림 2-23]의 M-file을 전역 변수를 이용하여 다시 작성 해 보자!

[풀이]

[그림 2-24]는 해당 code를 보여 주고 있다.



앞으로 여러분은 정적분을 계산할 때나 미분 방정식의 풀 때 전역 변수를 많이 이용하게 될 것이다.

```
function [avg,med]=newstats2(in)

%NEWSTATS  서브 함수를 이용하여 평균과 중간 값을 구하는 함수.
global n;

n=length(in);
avg=newmean(in);
med=newmedian(in);

%서브 함수들.
function a=newmean(in)
%평균을 계산 한다.
global n;

a=sum(in)/n;

function m=newmedian(v)
%중간 값을 계산한다.
global n;

w=sort(v);
if rem(n,2)==1
   m=w((n+1)/2);
else
   m=(w(n/2)+w(n/2+1))/2;
end
```

[그림 2-24] "newstats2.m"파일의 내용

나. 입·출력 매개 변수의 갯수를 제어하는 "nargin, nargout"함수 :

function mode M-files에서 "**nargin, nargout**"함수들은 얼마나 많은 입·출력 매개변수가 사용되고 있는 지를 알려 준다. 결국, 여러분은 "**nargin, nargout**"함수들을 이용하여 입·출력 매개변수에 따라 다른 일을 수행할 수 있는 함수를 작성 할 수 있다.


주의

입·출력 매개 변수의 갯수를 임의로 설정하는 것을 가능하게 하는 "**varargin, varargout**"함수는 M-file programming II에서 다룰 것이다.

[예제 8] [그림 2-24]에서 주어진 "newstats2.m"파일을 출력 변수가 하나 일 때는 평균을 반환하고, 둘 일 때는 평균과 중간 값을 반환하는 함수로 고치시오.

[풀이]

```
function [avg,med]=newstats3(in)

%NEWSTATS  서브 함수를 이용하여 평균과 중간 값을 구하는 함수.
global n;

n=length(in);

if nargout ==1
   avg=newmean(in);
else
   avg=newmean(in);
   med=newmedian(in);
end

%서브 함수들.
function a=newmean(in)
%평균을 계산 한다.
global n;

a=sum(in)/n;

function m=newmedian(v)
%중간 값을 계산한다.
global n;

w=sort(v);
if rem(n,2)==1
   m=w((n+1)/2);
else
   m=(w(n/2)+w(n/2+1))/2;
end
```

[그림 2-25] "newstats3.m"파일의 내용



필자의 생각으로는 본 강의를 듣는 여러분이 적어도 공업 수학의 기본적인 내용은 배웠으리라고 본다. 사실, 필자는 matlab을 matlab제작회사인 "MathWorks.com"에서 발행한 매뉴얼과 여러 관련된 전공 서적으로 공부하였다. 참고로, 필자의 전공은 신호 처리 중에서 초음파 진단기를 연구하였다. 그래서, 앞으로 나오는 예제들 중 약간은 신호처리, 통계학, 광학 등에 치중한 예제가 많을 것이다. 그리고, 약간은 어려운 내용도 있을 것이다. 혹시, 필자가 잘못 계산한 부분이 있다면 서슴지 말고 알려주기 바란다.

[예제 9] 벡터 X와 벡터 H에 대한 컨버루션(convolution)을 구하는 함수를 작성해 보시오.

[풀이]
컨버루션은 신호 처리 분야뿐만 아니라, 공학에서 기본적으로 많이 이용되는 공식이다. (식 2-1)은 컨버루션에 대한 공식이다.

$$y(n) = \sum_{k=-\infty}^{\infty} x(k)h(n-k) = \sum_{m=-\infty}^{\infty} h(m)x(n-m) \qquad (2\text{-}1\text{식})$$

만일, 시간 n에서, 벡터 X가 신호이고, 벡터 H가 시스템의 계수라고 하면 벡터 Y는 주어진 시스템의 응답이 된다. (식 2-1)에서 알 수 있듯이 컨버루션은 교환법칙이 적용된다. 이제, (식 2-1)을 반복 루프 없이, 작성 해 보자! 할 수 있겠는가?

아래에 필자가 작성한 code를 보지 말고 우선, 여러분이 작성을 해보고, 필자가 작성한 coding과 여러분이 작성한 coding의 속도를 앞에서 배운 "**profile**"함수로 비교해 보라!

물론, matlab이 지원하는 "**conv.m**"함수와 기능이 같아야 옳게 작성한 것이다.

전형적인 사용법(syntax) : **conv.m**
"**c=conv(a,b)**"
→벡터 **a**와 **b**의 컨버루션을 구한다.
→반환되는 출력 벡터 **c**의 길이, 즉, length(**c**)=length(**a**)+length(**b**)-1

```
function z=limconv(x,h)

%
%LIMCONV(X,H)     X와 H의 콘버루션(convolution)을 구하는 함수 이다.
%
%1998.1.30      By Lim Jong_Su
%

len_h=length(h);
len_x=length(x);
len=2*len_h+len_x-1;

N=len-len_h;

inv_x=zeros(len-1,1);
inv_x(len_h:N)=x(end:-1:1);


for i=0:N-1;
   z(i+1)=sum(inv_x(N-i:end-i).*h(:));
end
```

[그림 2-26] "limconv.m"파일의 내용

혹시, 여러분은 컨버루션 행렬(convolution matrix)라는 것을 들어 본적이 있는가?
matlab에서는 주어진 벡터의 컨버루션을 구하는데 두 가지 방법을 제공한다. 그 중한 가지가 앞에서 학습한 "conv.m"함수를 이용하는 것이고, 다른 하나는 "convmtx.m"함수로 컨버루션 행렬을 만들어 이용하는 것이다.

다음은 주어진 벡터 a와b에 대한 컨버루션을 구하는 방법을 비교한 것이다.

"conv.m"함수를 이용한 방법	"convmtx.m"함수를 이용한 방법
□x=1:3; ↲	□x=1:3; ↲
□y=2:4; ↲	□y=2:4; ↲
□c=conv(x,y); ↲	□n=length(x); ↲
	□c=x×convmtx(y,n); ↲


주의 "convmtx.m"함수를 이용하는 것보다는, "conv.m"함수를 이용하는 것이 더 능률적이다.

[예제 10] 주어진 벡터 X에 대해 출력 변수가 하나면 컨버루션 결과를 반환하고, 출력 변수가 두 개이면 컨버루션 결과와 공 분산(covariance)을 반환하는 함수를 작성하시오. 단, (식 2-2)는 공 분산을 구하는 공식이다.

$$C_{xy} = Cov(x,y) = E\{[x - m_x][y - m_y]^*\} = E\{xy^*\} - m_x m_y^* \quad (식\ 2\text{-}2)$$

여기서, E{X}는 벡터 X의 평균을 의미한다.
*는 해당 벡터의 복소 공액(complex conjugate)을 의미한다.

[풀이]
필자가 보기에는 (2-1식)을 해결한 독자라면 (2-2식)은 그리 어렵지 않을 것이라고 본다.

!참고 문헌:
아래에 열거한 참고 문헌은 매우 우수한 서적들이다.
전자공학을 공부하는 학생이라면 한 번쯤 볼만한 서적들이니 참고하기 바란다.

* 컨버루션 행렬 : "statistical digital signal processing and modeling" 1996년
저자: Monson H. Hayes, 출판사: John Wiley & Sons, Inc.

*공 분산(covariance) : "수리 통계학"1995년.
저자: 허 문열, 송 문섭, 출판사: 博英社

*컨버루션 공식:"Introduction to digital signal processing"1989년
저자: Johnny R. Johnson, 출판사:Prentice-Hall

[그림 2-27]은 필자가 나름대로 작성해 본 code이다. 여러분이 작성한 code와 비교해 보기 바란다. [그림 2-27]에 보여준 code는 너무 길어서 한 장에 표현이 안되므로 두 페이지에 걸쳐 제시했다.

```
function [con_mat,cov_mat]=concov(x,y,p)

%
%CONCOV   컨버루션 행렬과 공분산을 구하는 함수.
%
%1998.1.30   By Lim Jong_Su
%

if (nargin==3) & (nargout==1)
   con_mat=convm(x,p);
elseif  (nargin==3) & (nargout==2)
   con_mat=convm(x,p);
   cov_mat=covariance(x,y);
else
   error('입,출력 매개변수의 갯수가 맞지 않음!');
end

%%%
```

```
%%%
```

VIII. 다차원 배열(multidimensional array) : matlab5.0이후 버전에만 있음

기존의 Matlab4.2c이전 버전에서는 2차원 이하의 배열, 즉, 벡터와 행렬만이 연산 할 수 있는 대상이었다. 그러나, Matlab5.0이후 버전부터는 다차원 배열의 정의와 연산이 가능하게 되었다.

다차원 배열(multidimensional array)이란 2개이상의 차원을 가질 수 있는 배열로써, 기존에 여러분이 2차원 행렬에 대해 이용했던 방법을 그대로 다차원 배열에서도 이용할 수 있다. 게다가, Matlab은 다차원 배열을 만드는데 유용한 함수들을 제공한다. 그렇다면, 다차원 배열은 구체적으로 어디에 이용되는가?

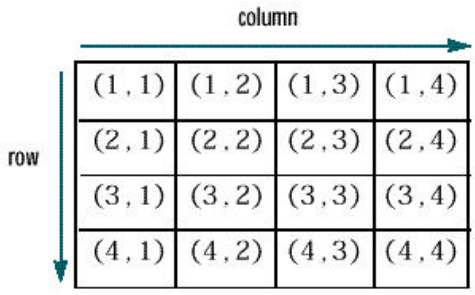
1. 2차원 데이터를 하나의 평면 또는 페이지(page)로 간주하고, 이들 페이지들로 이루어진 새로운 행렬에 대해 연산을 하고 싶은 경우
   예 : 이미지(image) 처리 등등

2. 다변수 또는 다차원 데이터를 이용해야 하는 항공역학이나, 온도 측정분야 따위.
   예 : 비행 중에 있는 날개의 각 요소에서 발생하는 압력을 측정하는 경우, 움직이는 물체를 추적하는 경우 등등

다차원 배열을 구성하는 각각의 요소로는 수치(numeric), 문자(character), 셀(cell), 구조체(structure) 등등, Matlab이 지원하는 모든 종류(class)의 데이터 type들이 될 수 있다.

가. 다차원 배열 :

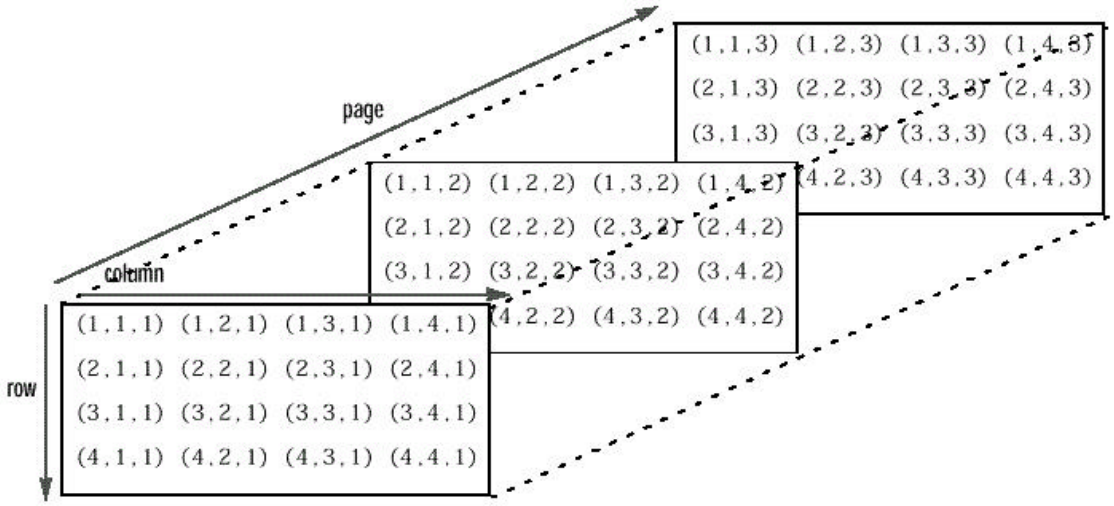
Matlab에서의 다차원 배열은 일반적인 2차원 행렬에 대한 확장이다.



[그림 2-28] 2차원 행렬의 예

[그림 2-28]은 2차원 행렬을 보여 주는데, 임의의 원소를 참조하기 위해서는 2개의 첨자(subscript)가 필요하다.

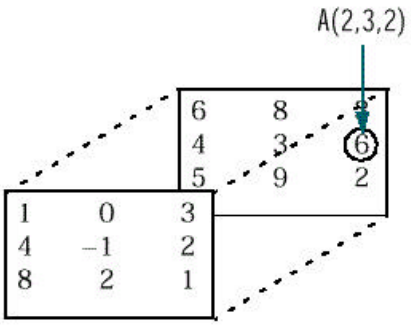
첫 번째 첨자는 행에 대한 index를 표현하며, 두 번째 첨자는 열에 대한 index를 표현한다. 다차원 배열의 경우에는 더 많은 첨자를 이용한다. [그림 2-29]는 3차원 배열을 보여 주고 있다.



[그림 2-29] 3차원 행렬의 예

3차원 배열은 3개의 첨자를 이용한다.
첫 번째 첨자는 "차원(dimension) 1"인, 행(row)에 대한 index를 표현하며, 두 번째 첨자는 "차원(dimension) 2"인, 열(column)에 대한 index를 표현하고, 세 번째 첨자는 "차원(dimension) 3"인, page에 대한 index를 나타낸다.

만일, 여러분이 크기가 3×3×2인 배열A에서 page2의 2행 3열의 원소를 참조하고 싶다면, [그림 2-30]에서 알 수 있듯이, (2, 3, 2)로 indexing하면 된다.




```
A(:,:,1) =

     1     0     3
     4    -1     2
     8     2     1

A(:,:,2) =

     6     8     3
     4     3     6
     5     9     2
```

[그림 2-30] 3차원 배열에서의 indexing방법

나. 배열(array)에서 indexing의 의미 :

Matlab은 실제 데이터의 차원(즉, 벡터는 1차원, 행렬은 2차원 등등)에 관계없이 주어진 배열을 "열 벡터(column vector)"의 형태로 저장한다. 예를 들어서, Matlab은 다음과 같은 행렬 A를 실제로는 열 벡터 A의 형태로 저장한다.

□A=[2 6 9;4 2 8;3 0 1]; ↲

$$\square\ A = \begin{bmatrix} 2 \\ 4 \\ 3 \\ 6 \\ 2 \\ 0 \\ 9 \\ 8 \\ 1 \end{bmatrix}$$

%실제로 행렬 A가 저장되는 모양!

즉, 행렬 A의 2행 3열원소인 "8"을 참조하기 위해서는 이전에 설명하였듯이 "**A(2,3)**"와 같이 할 수도 있지만, "**A(8)**"로 해도 2행 3열 원소를 참조 할 수 있다. 즉, Matlab은 임의의 크기를 갖는 배열이 주어지면, 그 배열을 열 벡터 형태로 바꾸어 저장한다. **이 개념은 Matlab의 indexing에서 아주 중요한 개념이다.** 또한, 이 개념은 다차원 배열에서도 그대로 적용된다.

만일, 여러분이 주어진 다차원 배열의 원소를 참조하기 위해 여러 개의 첨자들(subscripts)을 이용했다면, Matlab은 그 여러 개의 첨자를, 저장형태인 열 벡터에 대한 새로운 index를 계산한다. 여기서, 새로운 index는 주어진 다차원 배열에 할당된 배열의 크기에 기본을 두고 계산된다.

예를 들어서, 2차원 배열 A의 크기가 [d1 d2]로 주어졌다고 하자. 물론, d1은 배열 A의 행(row)의 수이고, d2는 배열 A의 열(column)의 수이다. 이제, 배열 A의 (i, j)원소를 참조하고 싶다고 하자. 그러면, Matlab은 우선 배열 A를 열 벡터로 저장하고, 배열 A의 (i, j) index는 새로운 index인 "**(j-1)*d1 + i**"로 바뀐 뒤, 실제로 저장된 열 벡터에서 "**(j-1)*d1 + i**"만큼 아래로 count하여 참조한다. 결국, "**(j-1)*d1 + i = offset**"의 개념이다.

앞에서 주어진 행렬A의 (2, 3)원소를 참조하기 위해서 Matlab은 우선, 열 벡터형태로 저장된 배열에 대해 "**offset**"을 계산하여, 그 수(즉, offset)만큼 아래로 count하여 참조한다. 즉, "(3-1)*3+2=8", 그래서, A(2, 3)원소는 A(8)로도 참조 할 수 있게 되는 것이다.

[예제 11] 행렬 A=randn(3,4)에서 A(7)을 A(i, j)의 형태로 표현한다면, i, j의 값은 무엇이 되겠는가?

전형적인 사용법(syntax) : **randn.m**
"**randn(m, n, p,...) or randn([m, n, p,...])**"
→정규 분포된 난수 배열(normally distributed random array)을 생성.
→입력 변수 벡터로 배열의 크기 결정.

[풀이]

우선, A는 열 벡터의 형태로 저장되므로, **offset**을 계산해야 한다.

행의 크기 : 3(즉, d1=3), 열의 크기 : 4(즉, d2=4), 또한, 7/3의 나머지는 1이므로, i=1이 된다. 그러므로, (j-1)*d1+i=(j-1)*3+1=7, j=3이 된다. 결국, A(7)은 A(1, 3)과 같다.

□A =randn(3,4) ↲

A =

0.0000	-1.8740	0.7310	0.6771
-0.3179	0.4282	0.5779	0.5689
1.0950	0.8956	0.0403	-0.2556

□A(7) ↲

ans =
0.7310

□A(1,3) ↲

ans =

0.7310

다. 다차원 배열의 생성 : indexing을 이용하는 방법

indexing을 이용하여 다차원 배열을 만들기 위해서는 먼저, 2차원 배열을 만들고, 그리고 나서, 그 2차원 배열을 확장한다. 예를 들어서, 2차원 배열 A가 다음과 같이 주어졌다고 하자.

□A =[5 7 8;0 1 9;4 3 6];↲

행렬 A는 3×3행렬이다. 즉, 2차원 행렬이다. 이제, 이 행렬A에 세 번째 차원을 첨가해 보자.

□A(:,:,2)=[1 0 4;3 5 6;9 8 7] ↲

A(:,:,1) =

5	7	8
0	1	9
4	3	6

A(:,:,2) =

1	0	4
3	5	6
9	8	7

□size(A) ↲

```
ans =
     3     3     2
```

결국, 원래 3×3**인 행렬**A**가** 3×3×2**인** 3**차원 배열**A**가 되었다**.

앞에서 배운 scalar expansion capabilities가 기억 나는가?
다차원 배열에서도 2**차원 행렬에서처럼** scalar expansion capabilities**가 이용 될 수 있다**.

□A(:,:,3)=5; ↲
□A(:,:,3) ↲

```
ans =

     5     5     5
     5     5     5
     5     5     5
```

그러므로, scalar expansion capabilities**는** colon **연산자와 함께**, **임의의 배열을 단일 값** (single value)**으로 채우는데 이용할 수 있다**. **이제**, 3**차원 배열** A**를 크기가** 3×3×3×2**인** 4**차원 배열**A**로 바꾸기 위해서는 어떻게 해야 하는지**, **우선**, **여러분이 생각해 보기 바란다**.
앞에서, 2**차원 행렬**A**를** 3**차원 배열**A**로 만들어 준 방법과 같은 방식으로 하면 된다**. **즉**,

□A(:,:,1,2)=[1 2 3;4 5 6;7 8 9];↲
□A(:,:,2,2)=[9 8 7;6 5 4;3 2 1];↲
□A(:,:,3,2)=[1 0 1;1 1 0;0 1 1];↲


주의 indexing**에 의한 방법으로 다차원 배열을 만드는 경우**, **첨가해주는 배열의 크기는 전체적인 배열의 모양을 이룰 수 있어야 한다**.

[예제 12] [**그림** 2-30]**에서 주어진 배열**A**의** (2, 3, 2)**원소를** A(i)**의 형태로 참조하고자 한다**. i**의 값을 구하시오**.

[풀이]
이번 예제는 [**예제** 11]**에 대한 일반화이다**.



간혹, 여러분 중에서는 그냥, 첨자 배열, 즉, A(2,3,2)로 참조하면 되지, 구태여, 까다롭게 A(i)형태로 참조하는 방법을 알아서 무엇하는가? 하는 의문을 갖을지도 모른다.
그러나, 만일, 여러분이 matlab에서 만든 데이터가 아닌 외부데이터(즉, 실험치, 다른 High-level언어로 만든 데이터 따위)를 "**fscanf.m**"등과 같은 함수로 파일 읽기를 행한다면, 입력 형식을 어떻게 정하던 간에 모든 데이터는 열 벡터형태로 읽혀진다. 그때, 읽혀진 열 벡터를 다시 원하는 임의의 배열로 만들기 위해서는 지금 설명하는 방법을 알아야한다. 사실, 이 방법은 다른 어떠한, 언어(예: C, Pascal, Basic 등등)에서 다차원 배열을 정의 할 때도 유사하게 적용될 수 있는 방법이다.

우선, 배열A는 3차원 배열이므로 최상위 차원인 1차원 page 벡터부터 고려해야 한다.
현재, 참조하고자하는 page벡터의 원소(즉, (2,3,**2**)에서 세 번째 원소인 "**2**")는 두 번째에 위치하므로 1을 빼준 값에 3*3을 곱해주어야 한다.

[문제1] 왜? 1을 빼주고 3*3을 해주어야 하는가?

그리고 한 단계 밑인 2차원 행렬에서 참조하고자하는 원소는 2행3열에 위치하므로 결국, i=(2-1)*3*3+(3-1)*3+2=17이 된다. 이제, 맞게 계산을 했는지, 확인해 보도록 하자.

```
□A =[1 0 3;4 -1 2;8 2 1];↲
□A(:,:,2)=[6 8 3;4 3 6;5 9 2] ↲

A(:,:,1) =

     1     0     3
     4    -1     2
     8     2     1


A(:,:,2) =

     6     8     3
     4     3     6
     5     9     2

□A(2,3,2) ↲
ans =

     6
```

□A(17) ↲

ans =

6

실행해본 결과, A(2,3,2)=A(17)이다. 그러므로, 옳게 계산 한 것이다.

이제, 여러분은 Matlab이 어떻게 데이터를 해당 workspace에 저장하는 지 알게 되었다. 또한, page의 개념도 알게 되었다.

일반적으로, 임의의 subscript(s1, s2, s3, ... ,sn)에 대해, 크기가 [d1 d2 d3 .... dn]인 배열(즉, n차원의 배열)의 원소를 참조하기 위한 "offset 공식"은 다음과 같다.

$$(s_n - 1)(d_{n-1})(d_{n-2}) \ldots\ldots (d_1) + (s_n - 1)(d_{n-2}) \ldots\ldots (d_1) + \ldots\ldots + (s_2 - 1)(d_1) + s_1 \quad \text{(2-3식)}$$

(2-3식)은 높은 차원에 대한 index를 먼저 계산하고 차례로 하위 index를 계산해 나가는 방식을 설명 해 주고 있다.

라. 다차원 배열을 만들어 주는 함수 :

Matlab은 다차원 배열을 만드는 데, 유용한 함수들을 제공한다.
그 중에서 특히 앞으로 많이 사용 할, **ones.m, zeros.m, rand.m, randn.m, repmat.m, cat.m, reshape.m**을 간단히 알아보자.

전형적인 사용법(syntax) : **ones.m**
"**ones(m, n, p,...) or ones([m, n, p,...])**"
→모든 원소가 "**1**"로만 이루어진 배열이다.
→입력 변수에 따라서, 생성되는 배열의 크기는 m×n×p×....이 된다.

"**zeros.m**"함수는 "**ones.m**"과 같은 기능을 하는데, 단지, 모든 원소가 "**0**"으로만 구성된다. "**randn.m**"함수가 정규 분포된 난수(normally distributed random number)를 발생하는 반면, "**rand.m**"함수는 균등하게 분포된 난수(uniformally distributed random number)를 발생하는 함수이다. 여러분도 알다시피, 난수에는 지수난수, 포아송 난수, 등 여러 가지 난수가 있다. 이들 난수는 신호 처리분야에서 잡음(noise)의 역할을 한다. Matlab의 toolbox 중에서 통계 toolbox에서는 난수를 분석하기 위한 많은 특수 함수들이 제공되며, 통신 toolbox에서는 simulink와 연결된 유용한 특수 함수가 제공된다. 필자의 생각으로는 양이 상당히 많지만, "통신"을 전공하는 분이라면 통신 toolbox를 한번쯤 공부하기를 권한다.

전형적인 사용법(syntax) : **repmat.m**

"**B=repmat(A, [m, n, p,...])**"

→**블록 배열**(block array)**A로 구성된 배열 B를 생성한다.**

→**블록 배열A는 임의의 차원을 가질 수 있다.**

→**작은 배열을 큰 배열로 만들고 싶을 때 이용.**

→**유사한 기능을 가진 함수로는 "kron.m"함수가 있다.**

사용 예 :

```
□a=[1 2;3 4] ↲
□b=repmat(a,[2,3]) ↲
b =

     1     2     1     2     1     2
     3     4     3     4     3     4
     1     2     1     2     1     2
     3     4     3     4     3     4

□c=kron(eye(2),a) ↲
c =

     1     2     0     0
     3     4     0     0
     0     0     1     2
     0     0     3     4
```

→**즉, "kron.m"함수는 eye(2)의 각 원소에 배열 a를 곱한 것과 같다.**

전형적인 사용법(syntax) : **cat.m**

"**B=cat(dim, A1, A2, A3, ...)**"

→**배열**A1, A2, A3, ...**를 차원이 dim인 새로운 배열** B**로 합성한다**.

→cat(1, A, B)**는** 1**차원 배열로 배열** A, B**를 합성하므로**

→ [A ; B]**와 같다**.

→cat(2, A, B)**는** 2**차원 배열로 배열** A, B**를 합성하므로**

→ [A B]**와 같다**.

사용 예 :

□a=[1 2;3 4] ↲

□b=[5 6;7 8] ↲

□c=cat(3, a, b) ↲

c(:,:,1) =

```
     1     2
     3     4
```

c(:,:,2) =

```
     5     6
     7     8
```

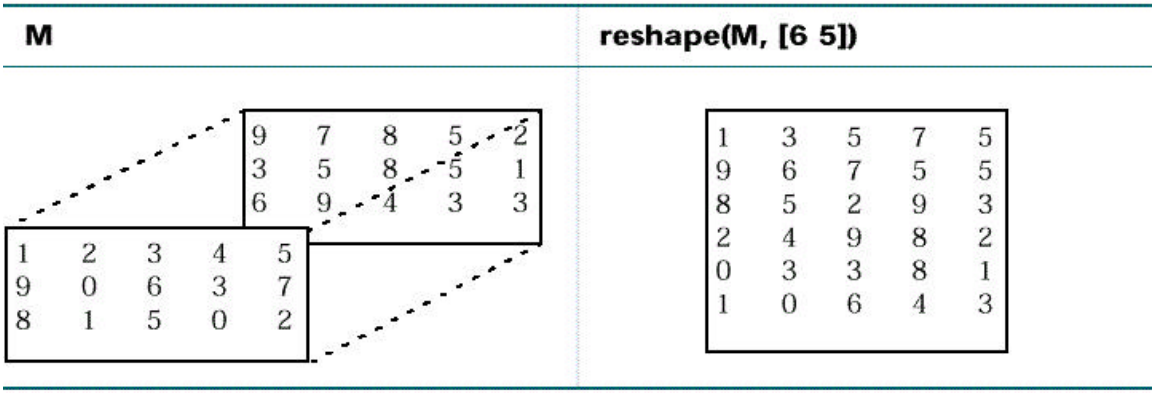
□d=size(c) ↲

d =

```
     2     2     2
```

→**결국**, "**cat.m**"**함수는 배열** a**와** b**를 합성하여** 3**차원 배열을 만들었다**.

만일, 주어진 배열 A**의 원소는 그대로 보존하면서 크기나 배열 형태를 바꾸고 싶다면** "**reshape.m**"**함수를 이용하라! 필자가 생각하기에 무척 유용한 함수이다**.
예를 들어서, 크기가 3×5×2**인** 3**차원 배열** M**이 있다고 하자, 이** 3**차원 배열을 크기가** 6×5**인** 2**차원 행렬의 형태로 바꾸고 싶다면** [**그림** 2-31]**에서 보여준 것처럼**, "**reshape(M,[6 5])**" **하면 된다**.

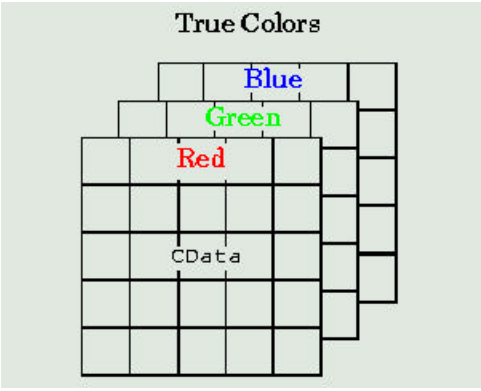


[그림 2-31] "reshape.m"함수의 이용 예

여기서, 중요한 것은 reshape된 배열의 원소의 수는 원래 배열의 원소의 수와 같아야 한다는 것이다.
[그림 2-31]에서 알 수 있듯이, "**reshape.m**"함수는 각각의 page에 대해 열 벡터방향으로 원소를 추출한다.

전형적인 사용법(syntax) : **reshape.m**
"**B=reshape(A, m, n, p,...) or B=reshape(A, [m, n, p,...])**"
→임의의 배열A를 차원이 m, n, p, ...인 새로운 배열 B로 재배치한다.

[예제 13] 다차원 배열에 대한 좋은 예제로 RGB image처리를 들 수 있다. Matlab은 하나의 RGB image에 대한 데이터를 [그림 2-32]에서 보여준 것처럼 3차원 배열에 저장한다.



[그림 2-32] RGB image 데이터

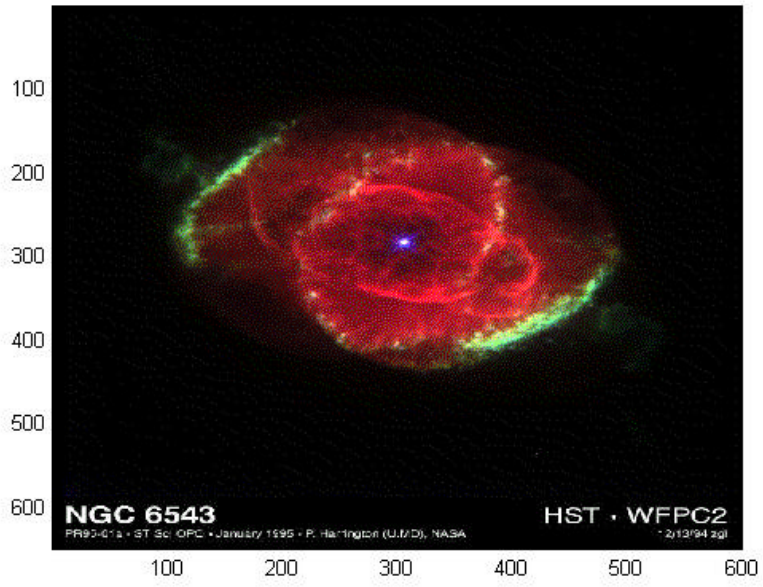
[그림 2-32]에서 CData란 각각의 pixel에 주어진 색채의 농도를 말한다.
Matlab은 여러분이 image를 공부하는데 유용한 여러 가지 image 데이터를 제공하고 있다. 그 중에서 "**ngc6543a.jpg**"그림을 예로 들어보겠다.

본 예제에서 "imread.m"과 "image.m"라는 함수를 이용할 것인데, 이 함수에 대한 구체적인 내용은 다음 단원인 "Matlab graphic"에서 다루겠다. 두 함수에 대해 간단히 요약 해보자.

전형적인 사용법(syntax) : **imread.m**
"B=reshape(파일이름.확장자)"
→파일에 있는 image데이터를 일어서 배열 B에 저장한다.

전형적인 사용법(syntax) : **image.m**
"image(image 데이터 C)"
→image 데이터인 배열 C를 screen에 image로 합성하여 보여준다.

```
□RGB=imread('ngc6543a.jpg'); ↲
□image(RGB)↲
```



[그림 2-33] "ngc6543a.jpg"의 모습

```
□size(RGB) ↲
ans =

    650    600      3
```

결국, [그림 2-33]을 표현하는 데이터 RGB는 [그림 2-32]에서 나타낸 것처럼 3차원 배열인 것을 알 수 있다.

IX. 문자열(string) :

가. 문자열의 정의 :

문자열(text strings)은 단일 인용부호(single quotes ‘, ’)로 둘러싸여 표현된다.


주의

C언어에서는 문자열을 표현하기 위해
이중 인용부호(double quotes “, ”)가 사용된다는데 주의!

예를 들어보면,

```
□s='Hello' ↲

s =

Hello
```

결국, 변수 s에 문자열 “Hello"가 저장된다.

이제, 문자열이 어떻게 변수에 저장되는지 알아보기 위해 변수의 크기를 알려주는 “size.m" 함수를 이용해보자.

```
□len_s=size(s) ↲

len_s =

     1     5
```

변수 len_s로부터 여러분은 matlab이 문자열을 행 벡터방향으로 저장한다는 것을 알 수 있을 것이다. 또한, 배열 len_s의 원소가 5개인 사실로부터 문자 하나 하나가 배열의 원소가 된다는 것도 알 수 있을 것이다.

matlab은 문자들을 해당하는 ASCII 값으로 바꾸어서 저장한다.
이 사실을 알아보기 위해, 입력 변수 값을 double형으로 바꾸어주는 “double.m"함수를 이용하자. 물론, 입력 변수 값을 char형으로 바꾸어주는 함수로는 ”char.m"이 있다.


주의

M-file programming II에서 여러분은 원하는 특성을 가진 새로운 데이터 타입을 정의하는 방법을 배우게 될 것이다. 이와 같은 새로운 data type을 user_defined data type이라고 하는데, “symbolic math.”에서 연산 대상이 새로운 데이터 타입인 “symbol”인 것을 예로 들 수 있다.

□s_val=double(s) ↲

```
s_val =

    72   101   108   108   111
```

배열 s_val**는 각각의 문자에 해당하는** ASCII **값을 얻었다.**

보통, matlab**에서 표현할 수 있는** ASCII **문자들은** $(32:127)_{10}$**범위에 있는 것들이다. 결국, 96개의 문자를 표현 할 수 있다.**

[**예제** 14] matlab**이 표현할 수 있는** 96**개의 문자를** 3×32**행렬로 나타내시오.**

[**풀이**]
ASCII code**값** 32:127, **즉,** 96**개의 원소를** 3×32**행렬로 나타내기 위해 앞에서 배운** "**reshape.m**"**함수를 이용한다.**

□F =reshape(32:127,32,3)'; ↲
□ascii=char(F) ↲

```
ascii =

  !"#$%&'()*+,-./0123456789:;<=>?
@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_
`abcdefghijklmnopqrstuvwxyz{|}~•
```

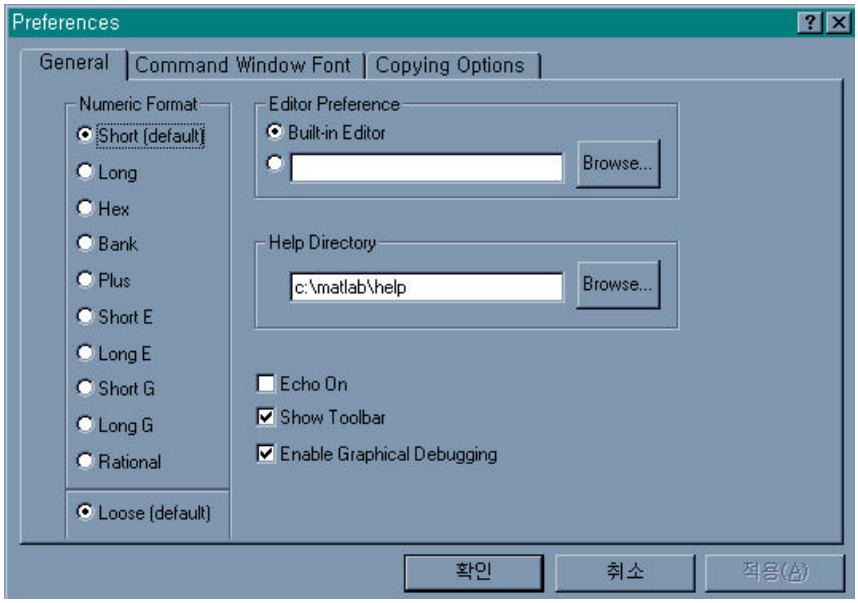

주의

matlab command window**의** toolbar**에 있는** **File****메뉴에서** **Preferences****를 선택하면** [**그림** 2-34]**와 같은** window**가 생성 될 것이다.**

General part**에 있는** **numeric format****은 지금 사용되고 있는** format**을 조정할 수 있게 해준다. 또한,** matlab**이 제공하는** "editor"**외에 다른** "editor"**를 사용할 수 있도록** option**을 제공한다.**

Command Window Font part**를** click**하면 현재 사용하고 있는** font**를 조정할 수 있게 해준다. 그러나, 여기서 설정하는** font**는 모든** command window**문자들에게 영향을 주므로 특수문자나** symbol**을 일정 부분에서만 이용하기에는 부적절하다.**

Copying Options part**에서는** clipboard format**을 설정하게 해준다.**



[그림 2-34] "**Preference**"의 사용 예

그렇다면, 수학 기호 또는, 그리스 문자는 표현할 수 없는가?
물론, 표현 할 수 있다. 여러 특수 문자와 심볼(symbol)들을 모두 표현 할 수 있는데, 사용 방법이 조금 다르다. 이것은 다음 단원(**3. matlab graphics**)에서 자세히 다루겠다.

나. 문자열의 합성 :

주어진 여러 개의 문자열을 합성하는 경우, "**square brackets**(즉, **[ ]**)"를 이용한다.

```
□x ='seoul' ↲
□y ='korea' ↲
□z =[x y] ↲
z =

seoulkorea
```

행 벡터 z에는 seoulkorea가 할당되는데, seoul과 korea에서 한 칸 띄우려면, seoul문자열 다음에 **spacebar**를 넣으면 된다. 즉,

```
□x ='seoul ' ↲     %문자열 "seoul"다음에 한 칸(spacebar) 띄우고 있다.
                   %여기서, spacebar(ASCII code 값:32)도 문자라는데 주의한다.
□z =[x y] ↲

z =

seoul korea
```

그런데, 만일, "□z=[x ; y] ↲ "하면 어떻게 되겠는가?

```
□z=[x;y] ↲

??? All rows in the bracketed expression must have the same
number of columns.
```

"error"가 발생한다. error 메시지를 보면, 행렬을 이루는 각 행의 열의 수가 틀리다는 것이다. 즉, 변수 "x"의 원소는 6개인데, "y"는 5개로 행렬의 모양을 이루지 못한다. 그러므로, 여러 개의 문자열을 합하여 행렬을 이루려면 각 행의 원소의 수가 같도록 spacebar로 맞추어 주어야 한다.

```
□x='seoul '; ↲   %len_x=length(x)는 "6"이다.
                 %즉, 변수 "x"는 6개의 원소를 갖는다.

□y='korea '; ↲   %len_y=length(y)는 "6"으로, spacebar를 첨가하였다.
                 %이제, 두 변수 x, y가 같은 길이를 가지므로
                 %행렬을 이룰 수가 있다.

□z=[x ; y] ↲     %z=['seoul ' ; 'korea ']와 같은 표현이다.

z =

seoul
korea
```

그러나, 이렇게 매번 spacebar로 문자열의 길이를 맞추어 주는 것은 어려운 일이다. 그래서, 문자열을 되돌려 주는 함수"char.m"을 사용한다. 이 함수는 각각의 행을 이루는 문자열의 길이가 같도록 spacebar를 자동 첨가 시켜준다.

```
□a='apple'; ↲
□b='orange'; ↲
□c='bananas'; ↲
□abc=char(a,b,c); ↲

abc =

apple
orange
bananas
```

이밖에도 cell, struct를 사용하는 방법도 많이 사용되는데, M-file programming.II에서 다루고자 한다.

다. 문자열로 표현된 수식의 연산 :

여기서 배울 "eval.m"함수와 "feval.m"함수는 나중에 여러분이 grahical user interface(GUI)용 프로그램을 작성하기 위하여 uicontrol과 uimenu를 사용하는 경우, 꼭 필요한 함수들이므로 특별히 잘 알아두기 바란다. 그래서, 여러 가지 예제를 다룰 것이다. 만일, 여러분이 GUI를 이용한 프로그램을 만들고 싶다면, 앞으로 제시할 예제들은 상당히 유용하게 사용 될 것이다.

현재 저의 홈페이지에서 제공하는 "lim97b.m (ver4.2c용)"함수는 사용자가 임의로 제공하는 수식 열을 그대로 실행할 수 있는데, 소스를 보면, "eval.m"함수가 있을 것이다. 이 함수가 주어진 수식 열(문자열)을 연산하는 부분이다.

1. eval.m 함수 :

"matlab의 수식, 문장, 함수 호출(function call)"을 포함하는 문자열(string)을 계산한다.

전형적인 사용법(syntax) : **eval.m**

(1) **a=eval('수식')**

→주어진 수식을 계산한 결과를 반환한다.

(2) **[a1, a2, a3 ...]=eval('수식') 또는,**

eval('[a1, a2, a3 ...]=수식')

→주어진 수식의 결과 값이 여러 개 일 때,

→출력 변수 a1, a2, ... 에 순서대로 반환한다.

(3) **[a1, a2, a3 ...]=eval('수식', '두 번째 수식')**

→만일, 주어진 수식에 문제가 없어서 잘 계산되면

→그 결과 값을 출력 변수 a1, a2, ... 에 순서대로 반환한다.

→ 만일, 주어진 수식에 문제가 있는 경우, 두 번째 수식을 계산하여

→ 그 결과 값을 출력 변수 a1, a2, ... 에 순서대로 반환한다.

우선, 각각의 경우에 대한 간단한 예제부터 다루어 본다.

```
□X=2+sin(90*pi/180); ↲      %matlab은 모든 각도를 radians로 표현한다.
                            %현재, 각도를 degree 단위로 바꾸었다.
□Y=3+log10(100); ↲          %"log10.m"은 상용 로그이고,
                            %"log.m"은 자연 로그이다.
□Z=eval('X+Y') ↲

Z =

     8
```

문자열(수식열) 'X+Y'를 "eval.m"함수가 계산하여, 그 결과를 변수 Z에 반환하고 있다.

```
□[row, col]=eval('size(rand(2,3))') ↲   %출력 변수가 한 개 이상인 경우.
row =

     2

col =

     3
□A=1:3; ↲
□B=2:4; ↲
□Z=eval('A*B', 'A.*B') ↲   %'A*B'는 연산이 안 되므로, 'A.*B'를 연산하고 있다.

Z =

     2     6    12
```

[예제 15] matlab은 파형을 발생시켜주는 여러 가지 함수를 가지고 있다. 이들 파형 발생 함수들은 대부분 시간 벡터를 필요로 한다. 예를 들어서, 1000[Hz] sampling frequency를 갖는 데이터를 만들기 위해서는, 우선 다음과 같은 시간 벡터(time vector)를 먼저 만들어야 한다.

```
□t=(0:0.001:1)'; ↲
```

벡터 "t"는 모두 1001개의 원소로 이루어진 열 벡터인데, 시간 구간은 0 1[sec]까지 1[msec]단위로 증가하는 경우에 해당한다.

[문제] 왜? 벡터 "t"를 열 벡터로 표현했는가? 이 문제는 matlab을 이용하여 신호처리를 하는 경우 아주 중요한 문제이다.

이제, 주어진 시간 벡터 "t"로 50[Hz]와 120[Hz]를 가진 2개의 정현파를 만들어 보자

```
□y=sin(2*pi*50*t)+2*sin(2*pi*120*t); ↲
```

새로 만들어진 벡터 "y"도 1001개의 원소로 이루어진 열 벡터이다. 이제, 신호 벡터 "y"에 통신에서 많이 이용되는 AWGN(additive white Gaussian noise)을 섞어보자.



주의 AWGN(additive white Gaussian noise)에 대해서는 "Bernard Sklar"가 쓴 "Digital Communications Fundamentals and Applications"의 page29 30 을 참고하기 바란다. 출판사는 "Prentice_Hall"이다.

```
□yn=y+0.5*randn(size(t)); ↲   %"randn.m"함수에 대해서는 앞에서 설명하였다.
```

잡음이 섞인 열 벡터 "yn"의 원소 중에서 처음 50개만, "plot.m"함수로 도시하여 보자.

전형적인 사용법(syntax) : **plot.m**
"**plot(x,y,'linestyle')**"
→"**x, y**"에 각각의 데이터를 넣는데, 같은 크기의 배열이어야 한다.
→'**linestyle**'에는 여러분이 원하는 선의 모양(style)과 색을 정하여
→지정한다.

□plot(t(1:50),yn(1:50)) ↲
□xlabel('time[msec]') ↲
□ylabel('진폭') ↲
□title(['주파수가 ' '50' ' ,' '120' '인 경우!']) ↲

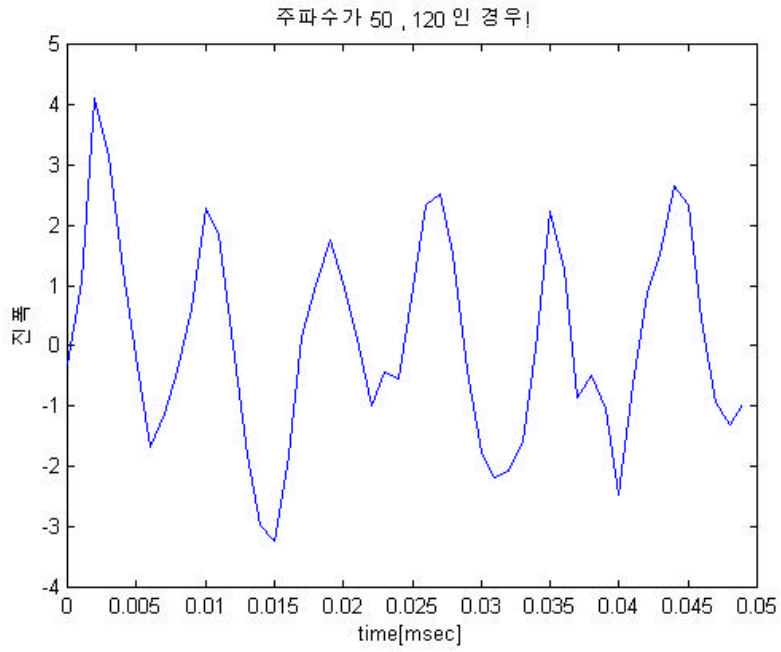
"**title.m**"함수의 입력으로는 모두 5개의 문자와 문자열이 square bracket(즉, **[ ]**)로 합성되어 하나의 행 벡터로 되었다.

전형적인 사용법(syntax) : **xlabel('문자열')**
"**xlabel('문자열').m**"
→임의의 모양으로 그려진 그래프에 주어진 '문자열'을
→ "**x**"축 라벨로 첨가한다.

마찬가지로, "**ylabel('문자열').m**"함수는 주어진 '문자열'을 "y"축 라벨로 첨가한다.

전형적인 사용법(syntax) : **title('문자열')**
"**title('문자열').m**"
→임의의 모양으로 그려진 그래프에 주어진 '문자열'을
→ "**title**"로 첨가한다.

[그림 2-35]는 결과를 나타내주고 있다.



[그림 2-35] [예제 15]의 실행 결과

[그림 2-36]은 [예제 15]의 내용을 정리한 M-file이다.
파일 이름은 "ex2_15.m"이라고 하였다.

```
%
%[예제 2-15]의 내용을 정리 한 M-file.
%
%1998.2.5          By Lim Jong_Su
%

clear all
close all

t=(0:.001:1)';      %시간 벡터 t.
f1=50;              %주파수 벡터 f1, f2
f2=120;

y=sin(2*pi*f1*t)+2*sin(2*pi*f2*t);
yn=y+0.5*randn(size(t));
plot(t(1:50),yn(1:50))
xlabel('time[msec]');
ylabel('진폭');
title(['주파수가 ' '50' ' ,' '120' '인 경우!'])
```

[그림 2-36] "ex2_15.m"파일의 내용

[예제 16] 이번 예제에서는 [예제 15]에서 알아본 내용 중, 변수 "y"를 계산하는 과정을 바꾸어 보고자 한다.

[그림 2-36]에 주어진 "ex2_15.m"파일에서 나타나는 수식 가운데
"y=sin(2*pi*f1*t)+2*sin(2*pi*f2*t);"을 문자열로 변환하여 "eval.m"함수로 계산할 것이다. 아주 간단한 기법이다. 그러나, 이 기법은 앞으로 여러분이 GUI프로그램을 작성 할 때 자주 이용하게 될 것이다.
우선, "num2str.m"함수와 "str2num.m"함수에 대해서 알아 둘 필요가 있다. 아주 중요한 함수들이므로 상세하게 설명하였다.

전형적인 사용법(syntax) : **num2str.m**

(1) **str = num2str(A)**
→배열 A를 문자열(string)로 변환 한 뒤 출력 변수 str에 반환한다.
→이 함수는 **labeling** 또는 **titling**을 할 때 유용하다.
(2) **str = num2str(A, 정밀도(precision))**
→배열 A를 문자열(string)로 변환 할 때, 주어진 정밀도에 따라
→출력 변수 str에 반환한다.
→default는 **4**개의 숫자로 표현하는 것이다.
(2) **str = num2str(A, 형식(format))**
→배열 A를 문자열(string)로 변환 할 때, 주어진 형식에 따라
→출력 변수 str에 반환한다.
→default는 "**%11.4g**"로 숫자를 표현하는 것인데,
→ 좀더 자세한 사항은 "파일 입·출력" 단원에서 다루겠다.

전형적인 사용법(syntax) : **str2num.m**

"**A = str2num('str')**"
→문자열(string) **str**을 matlab의 수치 표현으로 바꾼다.
→문자열 str은 수치(**numeric**)를 표현하는 것이어야 한다.

주어진 수식"y=sin(2*pi*f1*t)+2*sin(2*pi*f2*t);"에서 변수 f1, f2 그리고, 배열 t는 수식 이전에 정의되었다. 그러므로, 수식"y=sin(2*pi*f1*t)+2*sin(2*pi*f2*t);"을 문자열로 나타내면 "eval.m"함수로 계산 할 수 있다. 앞에서 이미, 배웠듯이, 문자열로 나타내려면 단일 인용부호를 좌우에 붙여주기만 하면 된다. 그리고, 변환된 문자열을 "eval.m"함수에 입력 변수로 넣어 주기만 하면 계산이 되는 것이다. 즉,

```
□y=eval('sin(2*pi*f1*t)+2*sin(2*pi*f2*t)'); ↲
```

이제, 여러분은 "ex2_15.m"파일의 내용 중에서, 문자열을 이용하여 "y"를 계산하도록 프로그램을 바꿀 수 있게 되었다. 또한, "title.m"함수의 입력부에 앞에서 설명한 "num2str.m"함수를 이용해 보자.

[그림 2-37]은 필자가 작성해 본 프로그램이다. 쉽게 이해하리라 본다.
파일이름은 "ex2_16.m"이라고 하였다.

```
%
%[예제 2-16]의 내용을 정리 한 M-file.
%
%1998.2.5          By Lim Jong_Su
%

clear all
close all

t=(0:.001:1)';      %시간 벡터 t.
f1=50;              %주파수 벡터 f1, f2
f2=120;

y=eval('sin(2*pi*f1*t)+2*sin(2*pi*f2*t)');
yn=y+0.5*randn(size(t));
plot(t(1:50),yn(1:50))
xlabel('time[msec]');
ylabel('진폭');
title(['주파수가 ' num2str(f1) ' , ' num2str(f2) ' 인 경우!']);
```

[그림 2-37] "ex2_16.m"파일의 내용

[예제 17] 간단한 예제를 하나 더 들어보았다. 이번 예제에서는 3개의 magic 행렬을 만드는 것이다.

```
%이름이 M1 부터 M3인 magic 행렬을 만다.

for n=1:3                                          %1번 line
   eval(['M' int2str(n) '=magic(n)'])              %2번 line
end
```

[그림 2-35] "ex2_15.m"파일의 내용

주어진 M-file에서 2번 라인을 잘 관찰해보자.

```
"eval(['M' int2str(n) '=magic(n)'])"
```

2번 라인에서, 'M'이라는 문자, 수치 n이 문자열로 변환된 문자, 그리고, 문자열 '=magic(n)', 이렇게 3개의 문자와 문자열이 하나의 문자열(행 벡터)로 합성(square brackets, "[ ]"에 의해서)된 것을 알 수 있다. 결국, 합성된 문자열이 문자열 연산 함수 "eval.m"에 의해 계산된다.

"n=1"인 경우, "eval.m"함수는 "M1=magic(1)"을 계산한다. 결국, 출력 변수 M1에 magic(1)의 값이 할당된다. 이런 방식으로 M3까지 계산한다. 여기서는, "eval.m"함수의 출력 변수들(M1 M3)이 입력 변수로 주어지고 있다. 또한, semicolon(즉, ;)이 없으므로 출력 값이 그대로 화면에 출력된다. 결과를 살펴보자.

```
□ex2_15 ↲

M1 =

     1


M2 =

     1     3
     4     2


M3 =

     8     1     6
     3     5     7
     4     9     2
```

이제, 여러분은 대충, "eval.m"함수의 사용방법과 문자열(string)의 개념을 잡을 수 있을 것이다.

```
□t=-2*pi:1/100:2*pi; ↲
□y='sin(t)*cos(t)'; ↲
□z=eval(y); ↲
```

결과는 어떻게 되겠는가?
쉽게 알 수 있듯이, "sin(t)*cos(t)"는 둘 다 행 벡터로 벡터 곱을 행할 수 없는 경우이다. 즉, error가 발생한다. 그러나, "sin(t)*cos(t)"를 원소 대 원소의 곱(즉, ".*")으로 바꾸어주면 계산이 가능해 진다.

□y='sin(t) .* cos(t)'; ↲
□z=eval(y); ↲

이처럼, 문자열로 표현된 수식을 계산하는 경우, 대부분 배열 연산보다는 원소 대 원소연산을 행하는 경우가 많다. 예를 들어서, 필자가 제작한 "**lim97b.m**"에 수식을 쓸 때도 원소 대 원소연산을 행하는 경우가 많다.
앞에서 이미 원소 대 원소연산에 대해서는 충분히 설명하였지만, 다시 간단히 [표2-3]에 그 종류를 나열하였다.

원소 대 원소연산	각 연산에 대한 설명
. *	원소 대 원소의 곱
. /	원소 대 원소의 나눗셈
. \	원소 대 원소의 왼쪽 나눗셈
. ^	원소 대 원소의 멱승(power)

[표 2 - 3] 원소 대 원소연산들

matlab5.0이후 버전에서는 이런 원소 대 원소연산 시 발생하는 "."문제를 해결하기 위해 "**vectorize.m**"함수를 제공하고 있다. 그 사용 예를 알아보자.

```
□exp='x*y+y/z^3'; ↲          %만일, x, y, z가 모두 행 벡터이면,
                              %당연히, 연산이 되지 않는다.
□exp_s=vectorize(exp) ↲

 exp_s =

x.*y+y./z.^3
```

결국, "**vectorize.m**"함수는 입력 문자열을 원소 대 원소연산으로 바꾸어 주어서 3개의 입력 벡터의 크기만 같으면, 원소 대 원소 연산을 할 수 있게 자동으로 "."을 적절하게 첨가시켜 준다.

```
□x=1:4; ↲
□y=2:5; ↲
□exp='x*y'; ↲
□result=eval(exp) ↲          %2개의 벡터가 모두 행 벡터이므로 곱셈이 안됨.

??? Error using ==> *
Inner matrix dimensions must agree.

□result_v=eval(vectorize(exp)) ↲

result_v =

     2     6    12    20
```

2. feval.m 함수 :

만일, 여러분이 어떠한, 함수, 즉, y=fun(x1, x2)의 기능을 하는 "fun.m"함수를 만들었다고 하자. 이 함수를 실행하는 방법은 많이 있겠지만, 여기서는 자주 사용되고 있는 2가지 방법을 보여주겠다.

```
□x1=a; x2=b; ↲
```

```
(a)□y=fun(x1, x2);  ↲        %지금까지 사용하여 온 방법!
(b)□y=feval('fun', x1, x2);  ↲
```

전형적인 사용법(syntax) : **feval.m**

"**[y1, y2, ...]=feval('함수 이름', x1, ... xn)**"

→주어진 함수의 입력 변수 값들(즉, **x1, ... xn**)로 함수를 계산하여,

→그 결과 값을 순서대로 출력 변수들(즉, **y1, y2, ...**)에게 반환한다.



다음 이 시간에 배울 내용 :

M-file **작성 시 유용한 함수들**(input.m, dialog**형 함수**, menu.m **등등**)**에 대해서 알 아 본다**.